

FGV EMap
João Pedro Jerônimo

Aprendizado Profundo

Revisão para A1

Rio de Janeiro
2026

Conteúdo

1	Segmentação Semântica	4
1.1	Introdução e Métricas	5
1.1.1	Métricas de Avaliação	5
1.1.2	Evolução das Abordagens	6
1.2	Ferramentas Fundamentais	8
1.2.1	Técnicas de Upsampling	8
1.2.1.1	Max Unpooling	8
1.2.1.2	Transpose Convolution	9
1.2.2	Mecanismos de Contexto e Campo Receptivo	10
1.2.2.1	Atrous Convolution	11
1.2.2.2	Image Pooling (Global Average Pooling)	11
1.2.3	Convoluções Eficientes	12
1.2.3.1	Atrous Spatial Pyramid Pooling (ASPP)	12
1.2.3.2	Convolução Normal V.S Separable Convolution & Atrous Separable Convolution	13
1.2.4	Blocos Residuais	15
1.3	Arquiteturas	15
1.3.1	SegNet	15
1.3.2	U-Net	16
1.3.3	ResUNet	16
1.3.4	DeepLab V1 & V2	18
1.3.5	PARSENet	19
1.3.6	PSPNet	19
1.3.7	Deeplab V3 & V3+	20
1.4	Percas	21
1.4.1	Cross Entropy Loss	21
1.4.2	Balanced Cross Entropy Loss	22
1.4.3	Focal Loss	22
1.4.4	Balanced Focal Loss	22
1.4.5	Loss Function for Regression	22
1.5	Pontos Práticos	23
1.5.1	A necessidade de Patches (Tiles)	23
1.5.2	O Problema do Contexto nas Bordas (Border Context Loss)	23
1.5.3	CNN Patch-wise Clássica vs. FCN Modela	24
2	Object Detection	25
2.1	Introdução	26
2.2	Métricas de Avaliação	26
2.3	Redes de Estágio Único (Single-Shot): A Família YOLO	27
2.3.1	Fundamentos do YOLO	27
2.3.2	Parametrização do vetor de saída	27
2.3.3	Caixas de Ancoragem (Anchor Boxes)	27
2.3.4	Supressão Não-Máxima (Non-Maximum Suppression - NMS)	28
2.3.5	Evolução Arquitetural	29
2.3.5.1	YOLOv1 & YOLO9000	29
2.3.5.2	YOLOv3	30
2.3.6	Loss Function	30

2.4	Redes de Dois Estágios e Segmentação de Instâncias: Mask R-CNN	31
2.4.1	Geração de propostas com a RPN (Region Proposal Network)	31
2.4.2	Alinhamento de Características com RoIAlign	32
2.4.3	Predições em Paralelo (R-CNN + FCN)	34
3	Recurrent Neural Networks (RNNs)	35
3.1	Introdução e Modelagem de Dados Sequenciais	36
3.2	Simple RNN (Vanilla RNN)	36
3.2.1	Arquitetura	38
3.2.2	RNN Unrolling	38
3.2.3	Tipos de mapeamento sequencial	39
3.3	Treinamento e Problemas de Gradiente na RNN	41
3.3.1	Backpropagation Through Time (BPTT)	41
3.3.2	A matemática dos gradientes explosivos ou desvanecentes	42
3.3.3	Métodos de mitigação	43
3.4	Limitações da RNN	44
3.5	Long-short Term Memory (LSTM)	44
3.5.1	A inovação no Fluxo do Gradiente	46
3.6	Gated Recurrent Unit (GRU)	46
3.7	Aplicando CNN	47
3.8	RNN Bidirecionais	48
4	Generative Adversarial Networks (GANs)	49

Segmentação Semântica

1.1 Introdução e Métricas

A segmentação semântica é uma tarefa de visão computacional que consiste em classificar cada pixel de uma imagem em uma categoria específica. Essa tarefa é fundamental para diversas aplicações, como direção autônoma, análise médica e realidade aumentada.

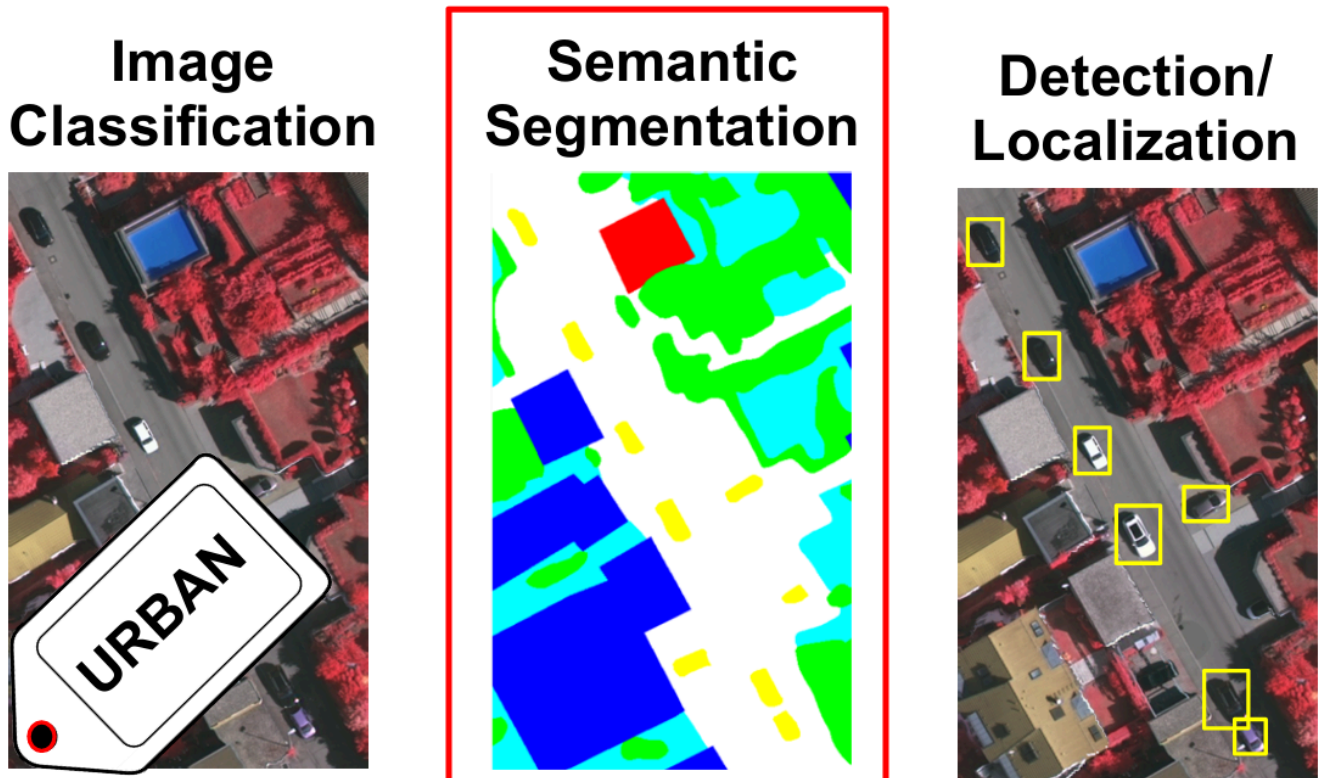


Figura 1: Áreas de visão computacional

Dentro do contexto de segmentação, vale diferenciar entre segmentação semântica e segmentação de instâncias. A segmentação semântica atribui uma classe a cada pixel, enquanto a segmentação de instâncias não apenas classifica os pixels, mas também distingue entre diferentes objetos da mesma classe.

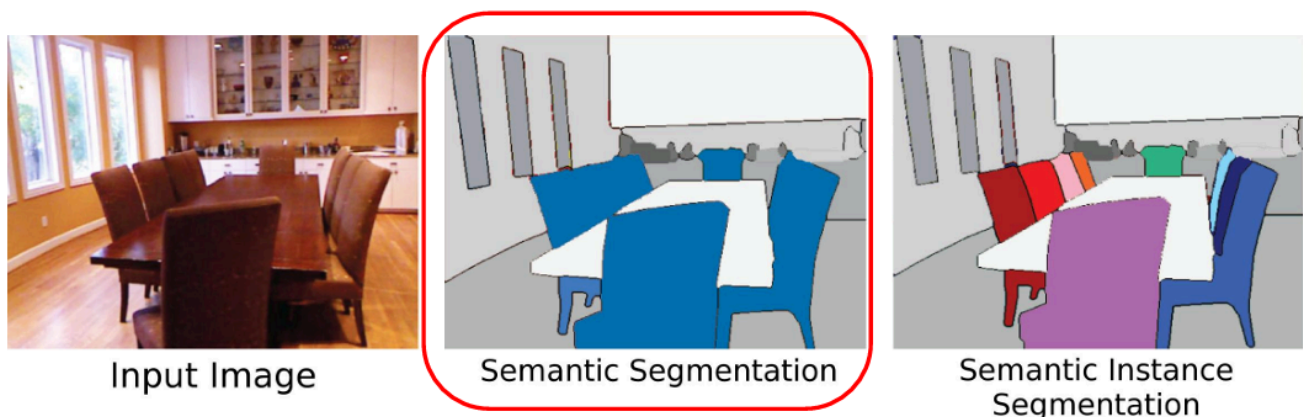


Figura 2: Diferença entre segmentação semântica e segmentação de instâncias

1.1.1 Métricas de Avaliação

Podemos utilizar algumas métricas para avaliar a performance de um modelo de segmentação semântica. As métricas mais comuns incluem:

Definição 1.1.1.1 (Intersection over Union (IoU)):

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \quad (1)$$

Definição 1.1.1.2 (Precision):

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

Definição 1.1.1.3 (Average Precision): Dado que na minha imagem eu tenho mapeado K classes, a métrica de Average Precision (AP) é definida como a média das precisões de cada classe:

$$\text{AP} = \left(\frac{1}{K} \right) * \sum_{k=1}^K \text{Precision}_k \quad (3)$$

1.1.2 Evolução das Abordagens

Vamos relembrar como é estruturada uma rede convolucional padrão para classificação de uma imagem.

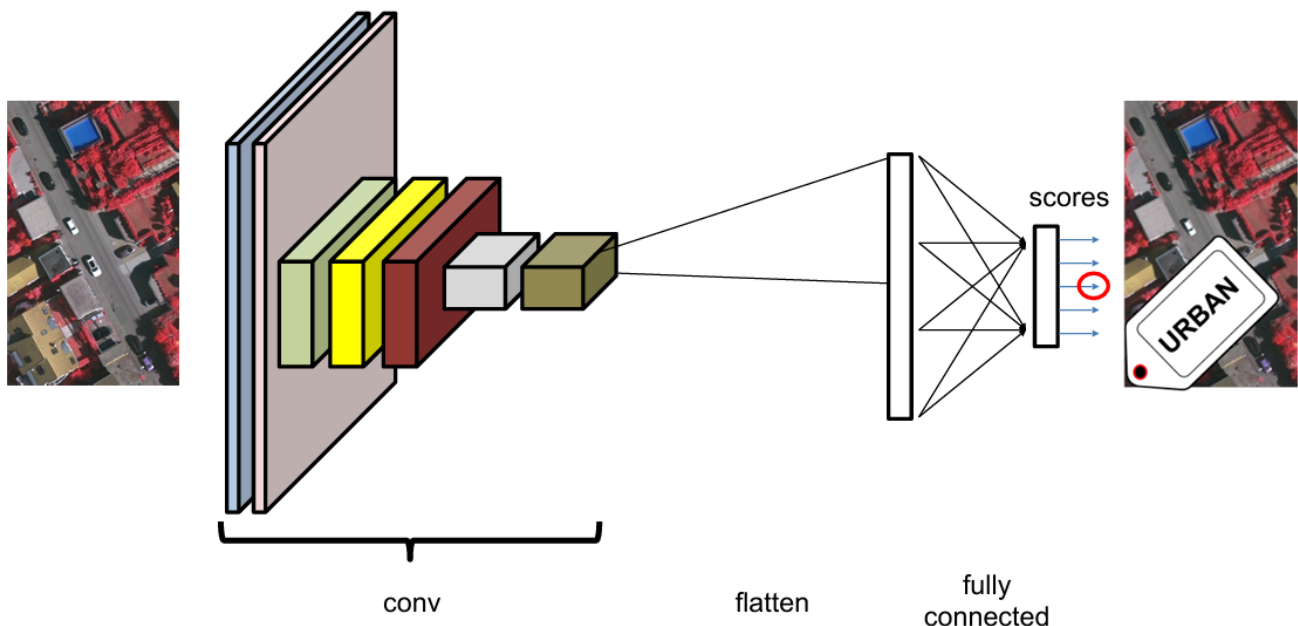


Figura 3: Estrutura de uma rede convolucional padrão

A imagem passa por uma série de canais de convolução, pooling e normalização, que extraem características relevantes. No final, temos uma camada totalmente conectada que produz a classificação final. Isso nos faz pensar em uma ideia, que tal termos uma janela que percorre a imagem e classifica cada pixel individualmente? Essa abordagem é conhecida como “sliding window” e é uma das primeiras tentativas de segmentação semântica.

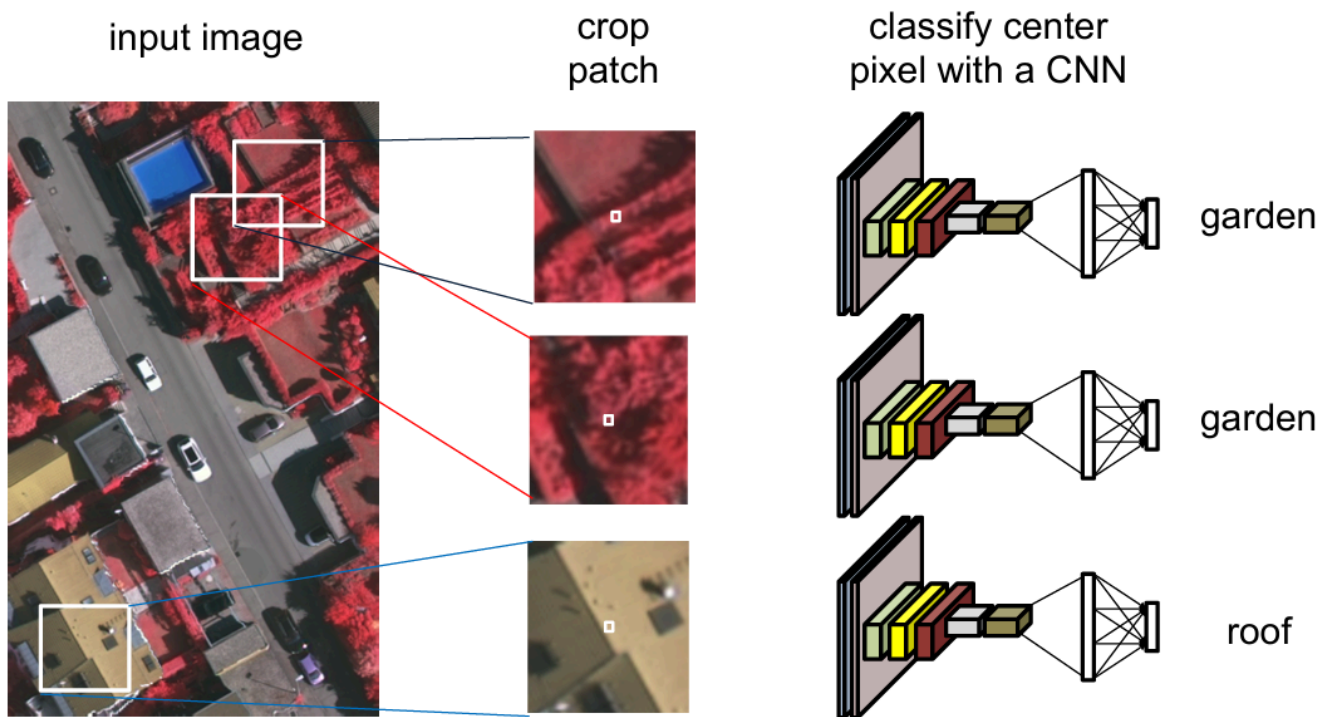


Figura 4: Abordagem de sliding window

No entanto, essa abordagem é computacionalmente cara e não aproveita o contexto global da imagem. Para superar essas limitações, surgiram as Fully Convolutional Networks (FCNs), que substituem as camadas totalmente conectadas por camadas convolucionais, permitindo que a rede produza mapas de segmentação diretamente.



Figura 5: Estrutura de uma Fully Convolutional Network (FCN)

Essa abordagem é interessante, já que permite que a rede aprenda a segmentar a imagem de forma mais eficiente, utilizando o contexto global e local. Além disso, as FCNs podem ser treinadas de forma end-to-end, o que simplifica o processo de treinamento. No entanto, as FCNs são MUITO pesadas, e por isso surgiu a ideia do **downsampling** e **upsampling** dentro da rede, onde basicamente diminuimos os tamanhos das features internas e depois vamos reconstruindo até o tamanho original da imagem. Essa abordagem é conhecida como **encoder-decoder**.

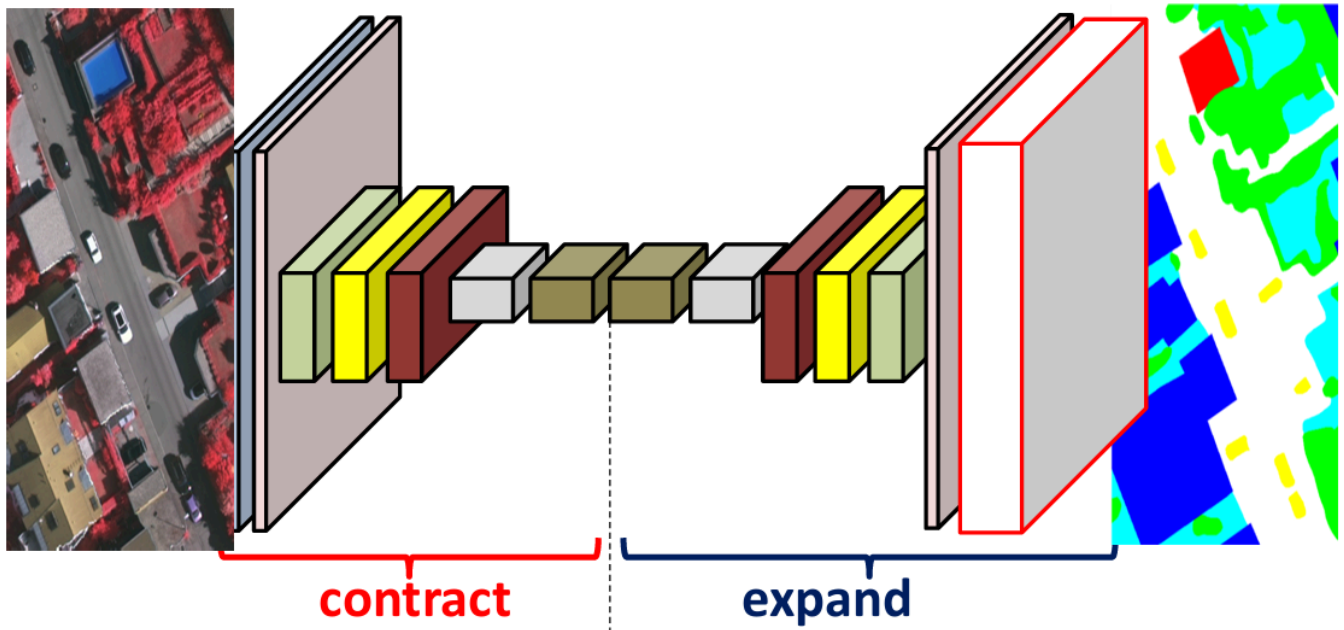


Figura 6: Estrutura de uma rede encoder-decoder

1.2 Ferramentas Fundamentais

Antes de irmos para as arquiteturas, precisamos relembrar algumas ferramentas fundamentais que são utilizadas em redes de segmentação semântica, vamos definir todas as ferramentas que iremos utilizar ao apresentar nossas arquiteturas, mas vale ressaltar que existem ferramentas que **caracterizam** algumas arquiteturas, como o **skip connection** na U-Net e os blocos residuais na ResUNet, mas isso não significa que essas ferramentas não podem ser utilizadas em outras arquiteturas, por exemplo, podemos utilizar blocos residuais na SegNet, mas isso não caracteriza a arquitetura como uma ResUNet.

1.2.1 Técnicas de Upsampling

O processo de upsampling é o processo de reconstrução da dimensão original das features internas da rede, que foram reduzidas pelo **downsampling**. Existem diversas técnicas para realizar o upsampling, como o **max unpooling** e a **transpose convolution**, que serão detalhadas a seguir.

1.2.1.1 Max Unpooling

Relembrando o que é o **pooling**, ele é uma operação que reduz a dimensionalidade das features internas da rede, geralmente utilizando operações como **max pooling** ou **average pooling**. O **unpooling** é o processo inverso, onde tentamos reconstruir a dimensão original das features a partir das features reduzidas. No entanto, o **unpooling** não é uma operação trivial, pois não temos informações suficientes para reconstruir a dimensão original de forma precisa.

O que acontece é que, **antes** de fazer o **pooling**, nós guardamos os índices dos valores máximos (no caso do **max pooling**), e depois utilizamos esses índices para reconstruir a dimensão original durante o **unpooling**. Essa abordagem é conhecida como **max unpooling**.

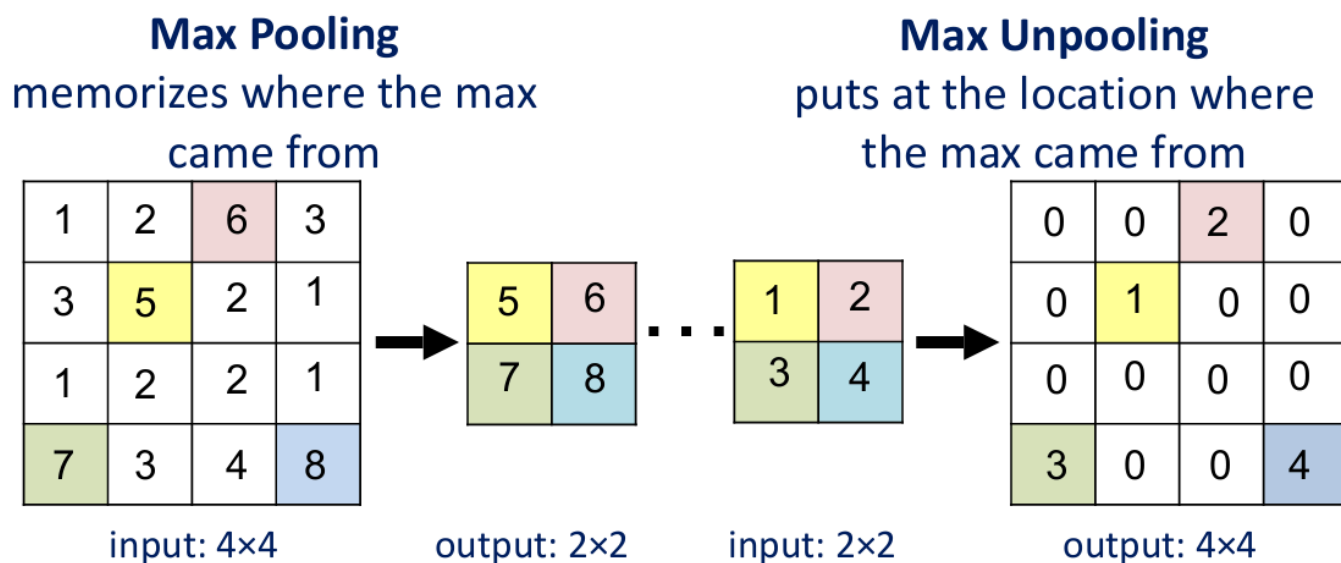


Figura 7: Exemplo de max unpooling

1.2.1.2 Transpose Convolution

A desvantagem do **max unpooling** é que ele depende dos índices dos valores máximos, o que pode limitar a capacidade da rede de aprender representações mais complexas. Uma alternativa é utilizar a **transpose convolution**, também conhecida como **deconvolution**. Essa operação é semelhante à convolução, mas ao invés de reduzir a dimensionalidade das features, ela aumenta.

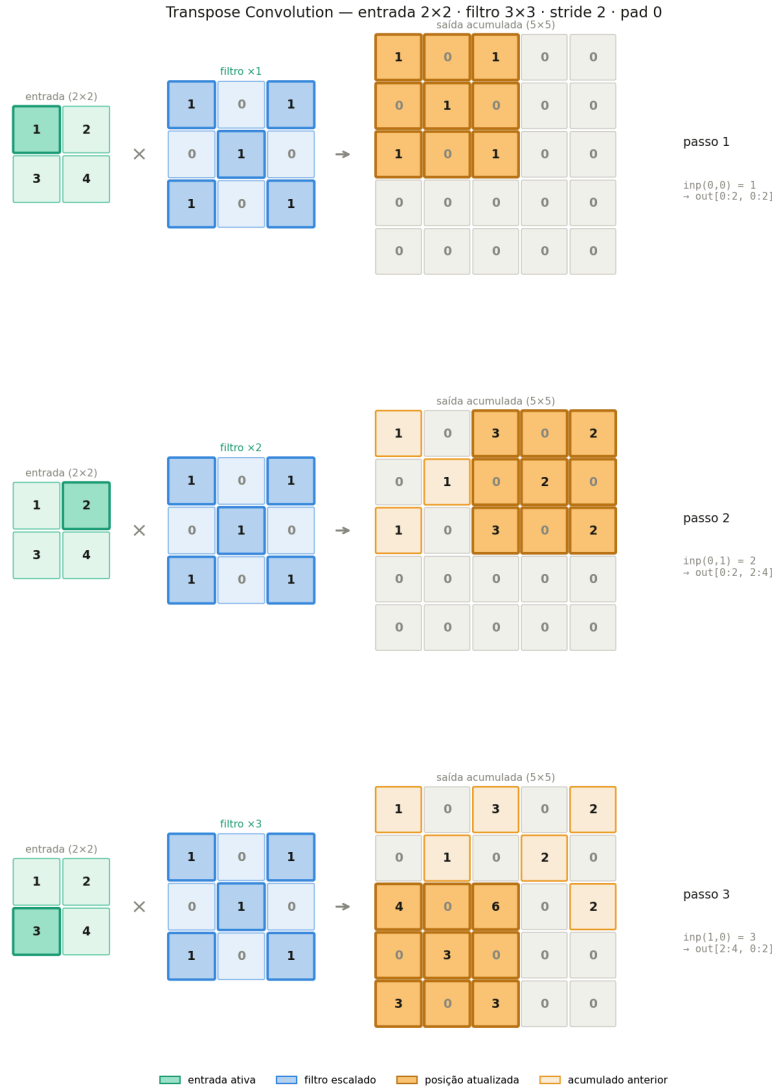


Figura 8: Exemplo de transpose convolution

Como vimos na disciplina de **machine learning**, podemos obter uma matriz de filtro chamada K_{col} a partir da operação *im2col*, que transforma a imagem em uma matriz de colunas. A **transpose convolution** é basicamente a operação inversa, onde aplicamos a matriz de filtro K_{col} na matriz obtida anteriormente

Se X é a entrada da convolução e K é o filtro tal que

$$Y = X * K \quad (4)$$

Já sabemos que

$$Y = K_{\text{col}} X_{\text{col}} \quad (5)$$

temos então que a transpose convolution é definida como

$$X_{\text{col}} = K_{\text{col}}^T Y \quad (6)$$

de tal forma que os X_{col} são reconstruídos a partir dos Y e do filtro K_{col} (não de forma perfeita pois há perda de informação na compressão do X para o Y , mas o objetivo é que a rede aprenda a reconstruir o X da melhor forma possível)

1.2.2 Mecanismos de Contexto e Campo Receptivo

Antes de irmos de fato para as arquiteturas, temos que definir um conceito usado em algumas delas, se não vamos interromper o raciocínio no meio do caminho. O conceito é o de **contexto**

global, que basicamente é a ideia de que, para classificar um pixel, precisamos levar em consideração não apenas os pixels vizinhos, mas também os pixels mais distantes da imagem. Só que usar filtros maiores consome mais memória, tempo de processamento e não permite que a rede aprenda a extrair características mais complexas.

1.2.2.1 Atrous Convolution

Para resolver esse problema, surgiu a ideia de **atrous convolution**, que é uma técnica que permite aumentar o tamanho do filtro sem aumentar o número de parâmetros da rede nem redimensioná-la. A ideia é inserir “buracos” (ou **holes**) entre os pixels do filtro, permitindo que ele “veja” mais pixels da imagem sem aumentar o número de parâmetros.

Definição 1.2.2.1.1 (Atrous Convolution): Dado um filtro K de tamanho $k \times k$, e uma feature map X , a **atrous convolution** é definida como

$$Y(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X(i + r * m, j + r * n) K(m, n) \quad (7)$$

onde r é o **rate** de atrous convolution, que determina o espaçamento entre os pixels do filtro.

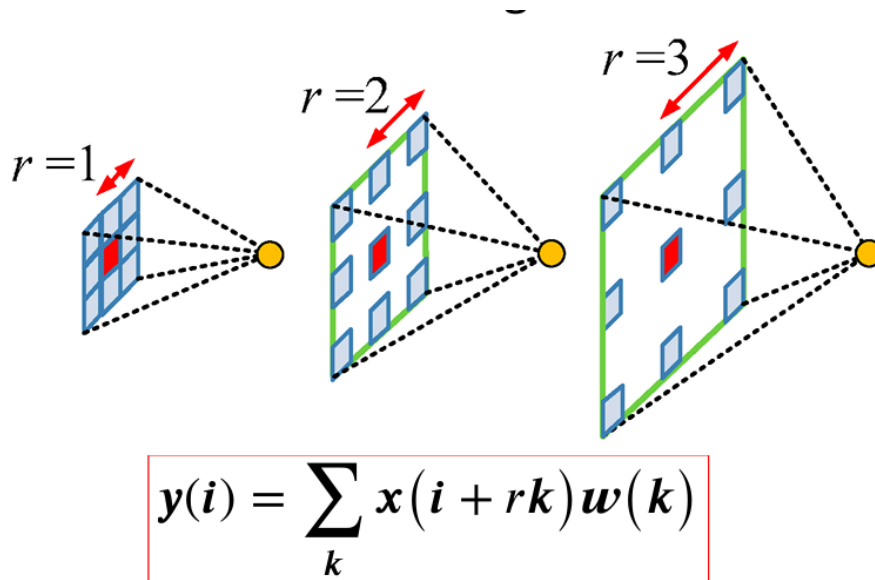


Figura 9: Exemplo de atrous convolution

1.2.2.2 Image Pooling (Global Average Pooling)

Essa é uma abordagem que permite trazer contexto global da imagem para a rede. Pegamos um feature map e aplicamos um average pooling em toda a imagem, obtendo um vetor de características que representa a imagem como um todo. Esse vetor é então redimensionado através de upsampling e concatenado com o feature map original, permitindo que a rede utilize informações de contexto global para melhorar a segmentação.

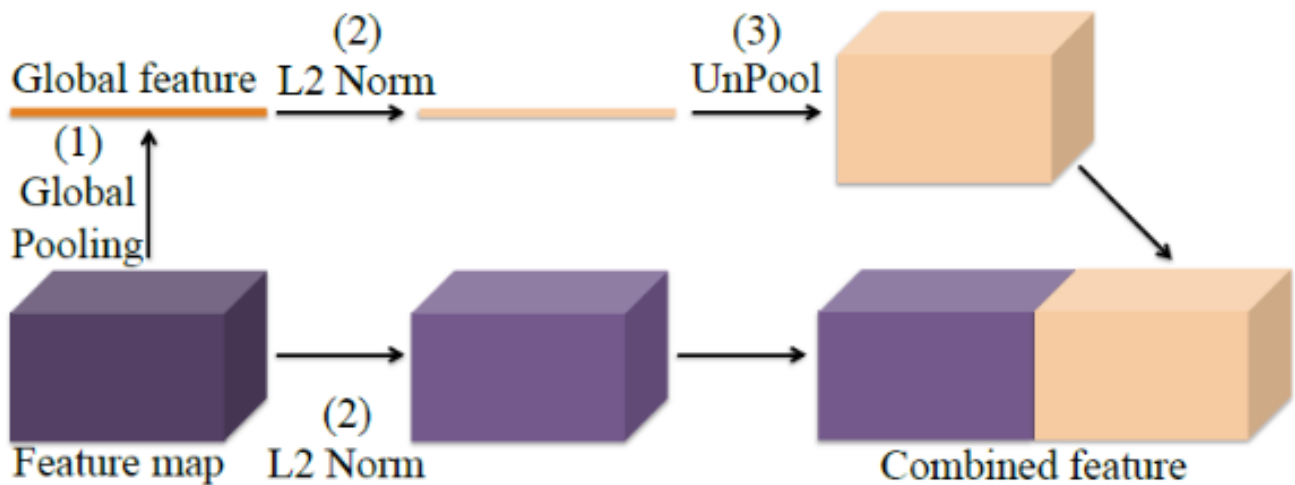


Figura 10: Exemplo de image pooling

1.2.3 Convoluções Eficientes

Como já discutimos n vezes, as convoluções são operações matemáticas bem caras, e por isso estamos sequer discutindo essas arquiteturas, pois elas são pensadas não somente para a melhor classificação da rede, mas para que possam ser eficientes!

1.2.3.1 Atrous Spatial Pyramid Pooling (ASPP)

Esse conceito é bem simples, vimos anteriormente sobre as Atrous Convolution, convoluções espaçadas para obtenção de contexto local da imagem. No entanto, esse passo pode ser tanto benéfico quanto maléfico! Imagine que a feature que queremos identificar é muito grande no conjunto das imagens, se a rate for muito pequena, isso pode prejudicar que os filtros consigam capturar essa feature, e o mesmo vale para features muito pequenas, se a rate for muito grande, isso pode prejudicar que os filtros a visualizem. A ideia do ASPP é justamente utilizar múltiplas rates de atrous convolution, para que possamos capturar features de diferentes tamanhos na imagem, de forma que elas rodem **paralelamente** e depois sejam concatenadas, permitindo que a rede utilize informações de diferentes escalas para melhorar a segmentação.

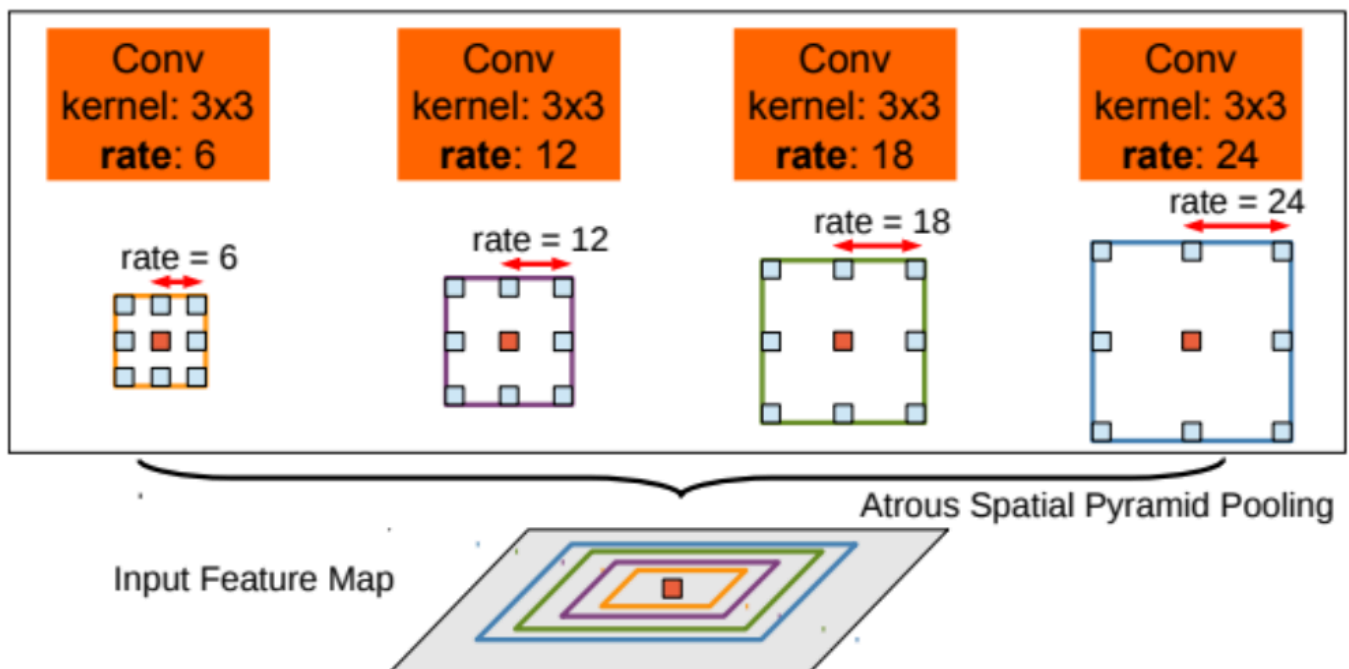


Figura 11: Exemplo de atrous spatial pyramid pooling

1.2.3.2 Convolução Normal V.S Separable Convolution & Atrous Separable Convolution

Em uma convolução normal, cada filtro é aplicado a todos os canais da imagem, o que resulta em um grande número de operações. Veja a imagem a seguir, por exemplo

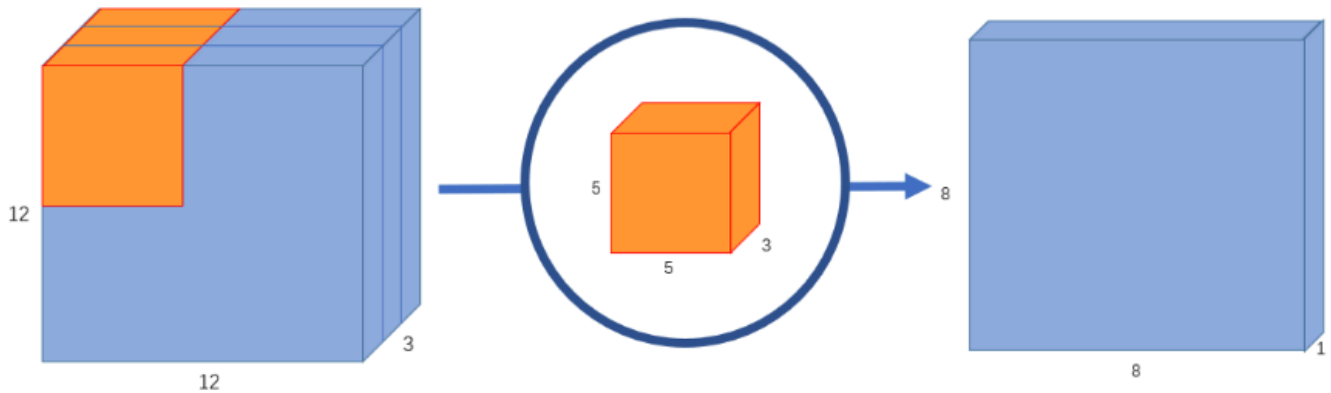


Figura 12: Exemplo de imagem com 3 canais

perceba que, ao aplicar o filtro na imagem com 3 canais, toda a informação foi sumariada em um único canal (assim, se quiséssemos várias feature maps, nós aplicaríamos C filtros diferentes para obter C feature maps). Como nosso filtro é $5 \times 5 \times 3$, temos um total de 75 parâmetros. Como a imagem é de tamanho 12×12 , o filtro vai percorrer a imagem 8×8 vezes, resultando em um total de $75 * 64 = 4800$ operações. Agora imagine o cenário que falei de utilizarmos múltiplos filtros, vamos usar por exemplo $C = 256$

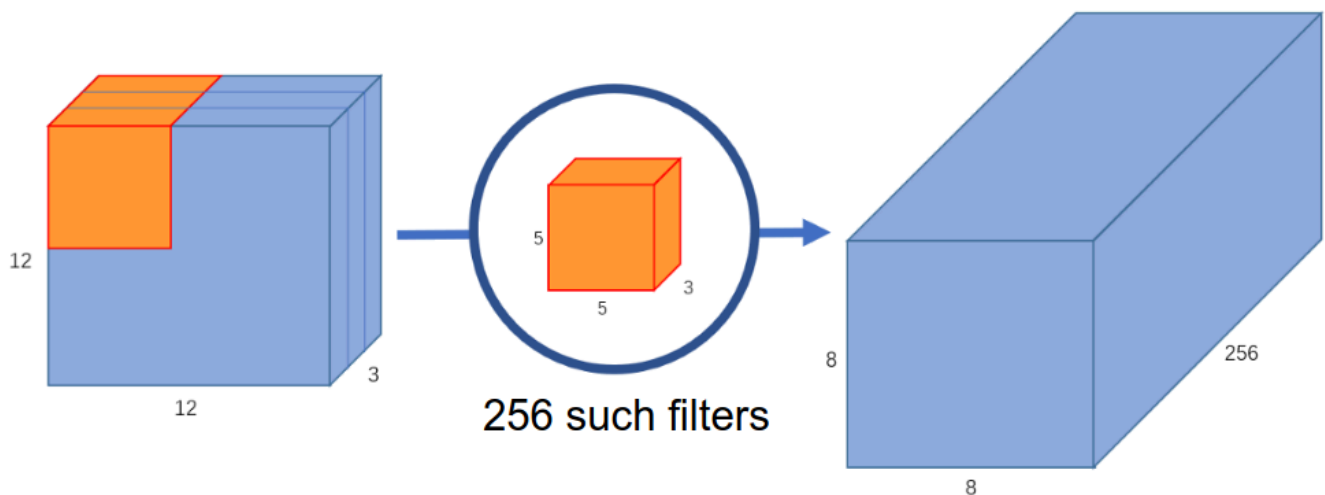


Figura 13: Exemplo de imagem com 3 canais e 256 filtros

Nesse exemplo, temos 256 filtros de tamanho $5 \times 5 \times 3$, resultando em um total de $75 * 256 = 19200$ parâmetros. Como a imagem é de tamanho 12×12 , o filtro vai percorrer a imagem 8×8 vezes, resultando em um total de $19200 \cdot 64 = 1.228.800$ operações. Isso é muito caro computacionalmente, e por isso surgiram as convoluções separáveis.

Agora vamos entender como funciona o processo da **SEPARABLE CONVOLUTION**, ela é dividida em 2 etapas, a **depthwise convolution** e a **pointwise convolution**. Na **depthwise convolution**, cada filtro é aplicado a apenas um canal da imagem, resultando em múltiplos feature maps, um para cada canal

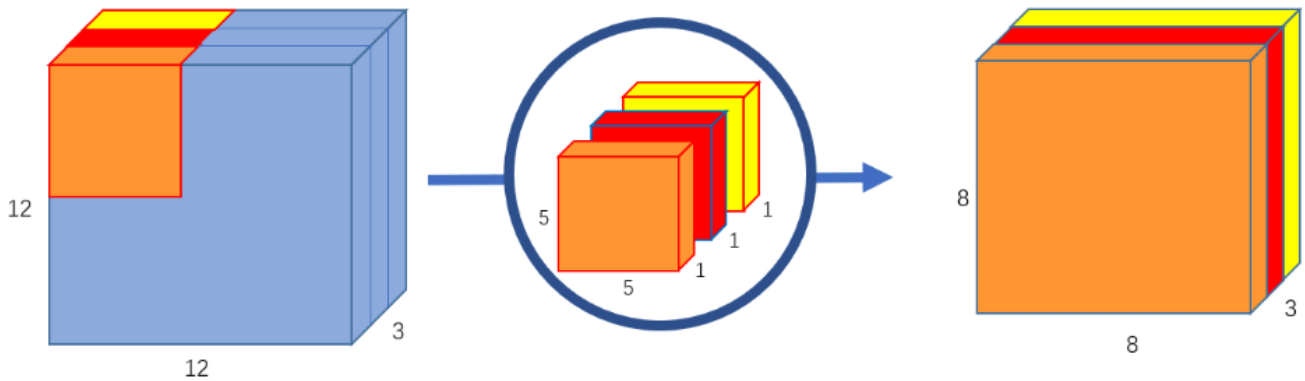


Figura 14: Exemplo de depthwise convolution com 3 canais

ainda existem os mesmos 75 parâmetros, já que são 3 filtros de tamanho 5×5 , mas agora temos 3 feature maps, um para cada canal. Agora, na **pointwise convolution**, aplicamos um filtro 1×1 em cada feature map, resultando em múltiplos feature maps, um para cada filtro (no mesmo estilo que o filtro antigo era aplicado na convolução normal)

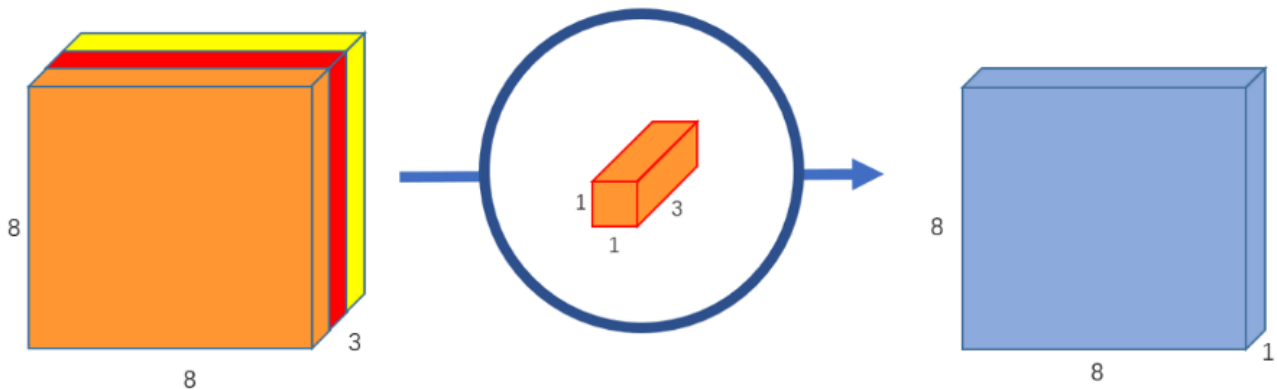


Figura 15: Exemplo de pointwise convolution com 3 canais

Agora temos que nosso número de parâmetros, além dos 75 do filtro anterior, temos mais esse filtro menor, nos resultando em 78 parâmetros. O número de multiplicações, na primeira etapa, é $75 \times 8 \times 8 = 4.800$, adicionando com a segunda etapa, temos $4800 + (1 \times 1 \times 3) \times 8 \times 8 = 4.800 + 192 = 4992$. No entanto, para obtermos múltiplos feature maps, vamos utilizar $C = 256$ filtros dos 1×1 que comentamos, dessa forma:

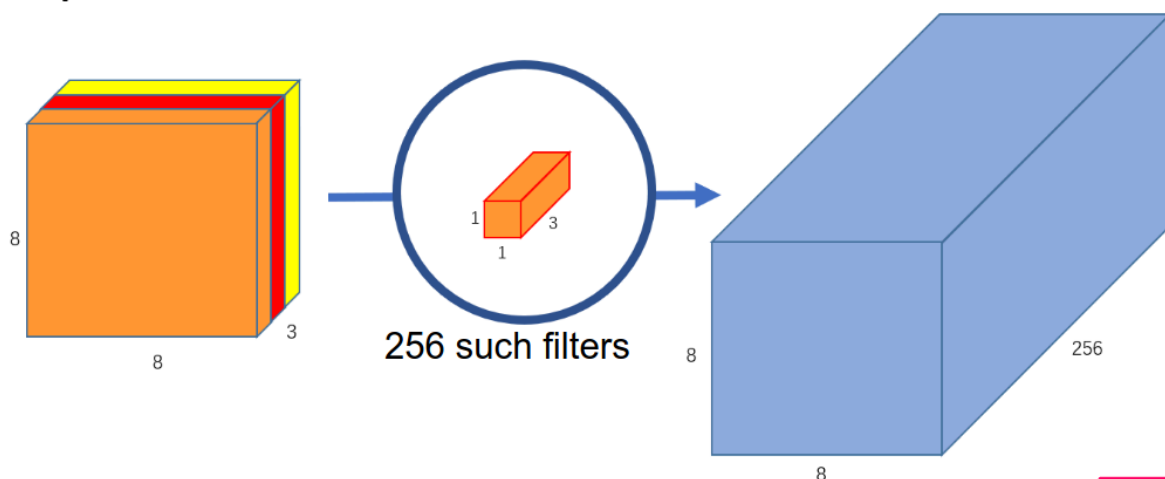


Figura 16: Exemplo de pointwise convolution com 3 canais e 256 filtros

Agora vamos ter que a quantidade de filtros é o filtro inicial mais os 256 outros filtros, logo: $75 + (1 \times 1 \times 3) \times 256 = 75 + 768 = 843$ parâmetros. E como aumentamos apenas os filtros 1×1 para obter os nossos 256 feature maps, o número de multiplicações passa a ser $4800 + (1 \times 1 \times 3) \times (8 \times 8) \times 256 = 4800 + 49152 = 53952$.

Esse conceito pode ser expandido para as convoluções Atrous, de forma que as mudanças necessárias são mínimas, mantendo tracking do rate r conseguimos aplicar o mesmo conceito de **depthwise** e **pointwise** para as convoluções Atrous, resultando em uma redução significativa no número de parâmetros e operações, mantendo a capacidade da rede de capturar informações de diferentes escalas.

1.2.4 Blocos Residuais

Normalmente, redes neurais são straight-to-the-point, nós temos a entrada x e a partir disso a rede modela uma função complexa F tal que

$$y = F(x) \quad (8)$$

no entanto, pode existir casos em que y é MUITO parecido com x com leves ajustes, e isso, surpreendentemente, pode dificultar muito o aprendizado da rede. Para consertar isso, os chamados **blocos residuais** foram introduzidos, de forma que a rede não aprende a relação direta entre x e y , mas sim a **diferença** entre eles (o quão diferente y é de x), ou seja, a rede aprende uma função F tal que

$$y = F(x) + x \quad (9)$$

O principal motivo dessa abordagem é o **gradiente no backpropagation**. Em redes comuns de deep-learning, o gradiente pode se tornar muito pequeno (ou até mesmo zero) à medida que é propagado para trás, dificultando o aprendizado. Com os blocos residuais, o gradiente pode fluir diretamente através da conexão de atalho, permitindo que a rede aprenda mais facilmente.

1.3 Arquiteturas

1.3.1 SegNet

A SegNet é uma arquitetura de rede neural convolucional projetada para segmentação semântica. Ela segue a arquitetura padrão que já demonstramos utilizando do método de max unpooling para realizar upscaling

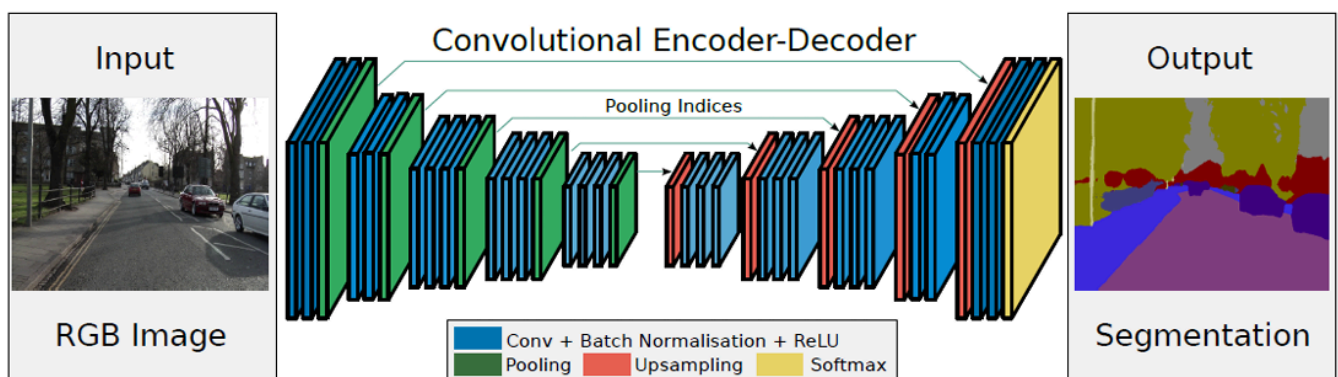


Figura 17: Arquitetura da SegNet

1.3.2 U-Net

Já na U-Net, a arquitetura é um pouco diferente, ela utiliza **skip connections** para conectar as camadas de downsampling com as camadas de upsampling, permitindo que a rede utilize informações de diferentes níveis de abstração para melhorar a segmentação.

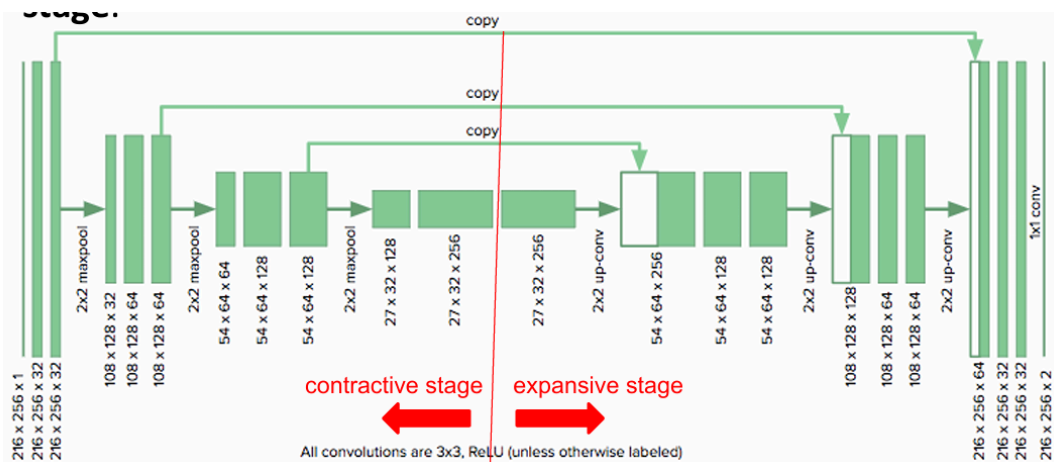


Figura 18: Arquitetura da U-Net

Nas camadas de upsampling, a U-Net utiliza **transpose convolution** para aumentar a dimensionalidade das features unida com um **aumento** nos canais das features. Após o transpose convolution, a U-Net concatena as features da camada correspondente de downsampling, permitindo que a rede utilize informações de diferentes níveis de abstração para melhorar a segmentação, como se ela falasse: “depois de reconstruir a imagem, eu obtive o seguinte mapa de feature, mas lá atrás antes de eu ter feito o downsampling, eu tinha obtido o seguinte mapa de feature, então vou juntar os dois para melhorar a segmentação” (por exemplo, se eu tenho uma imagem 32x32 na escala de cinza, com apenas um canal de cor, na hora do último upsampling, a camada logo após a transpose convolution terá 2 canais “de cor”, que seria o mapa obtido pela rede anteriormente e o mapa obtido na camada de upsampling).

1.3.3 ResUNet

Na ResUNet, a arquitetura é uma combinação da U-Net com blocos residuais, permitindo que a rede aprenda a diferença entre as features de downsampling e upsampling, melhorando ainda mais a segmentação.

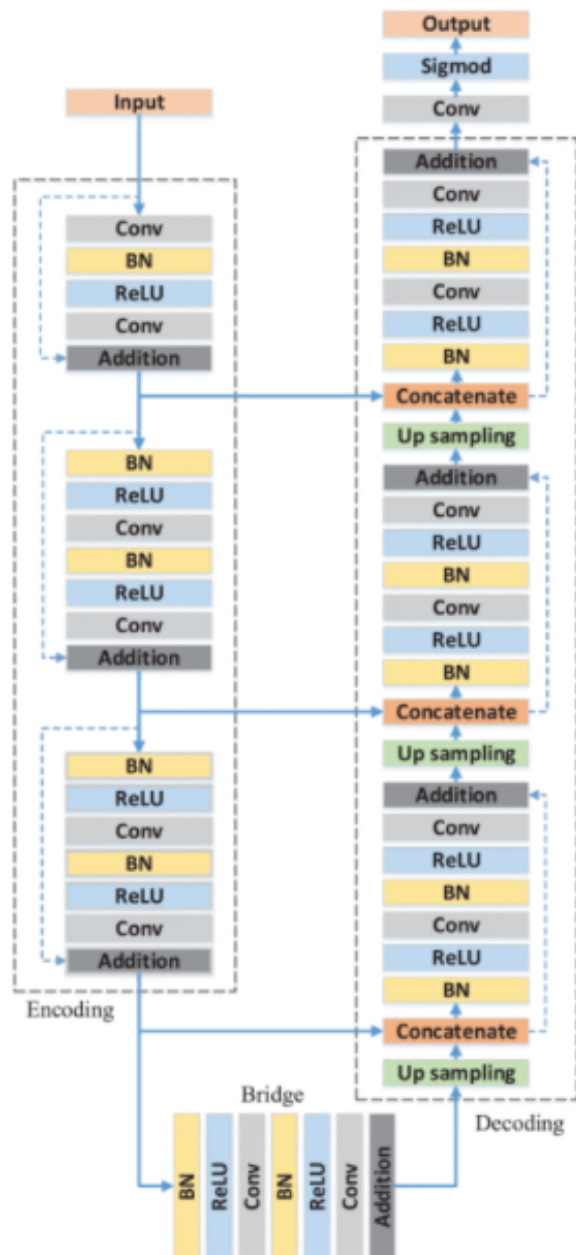


Figura 19: Arquitetura da ResUNet

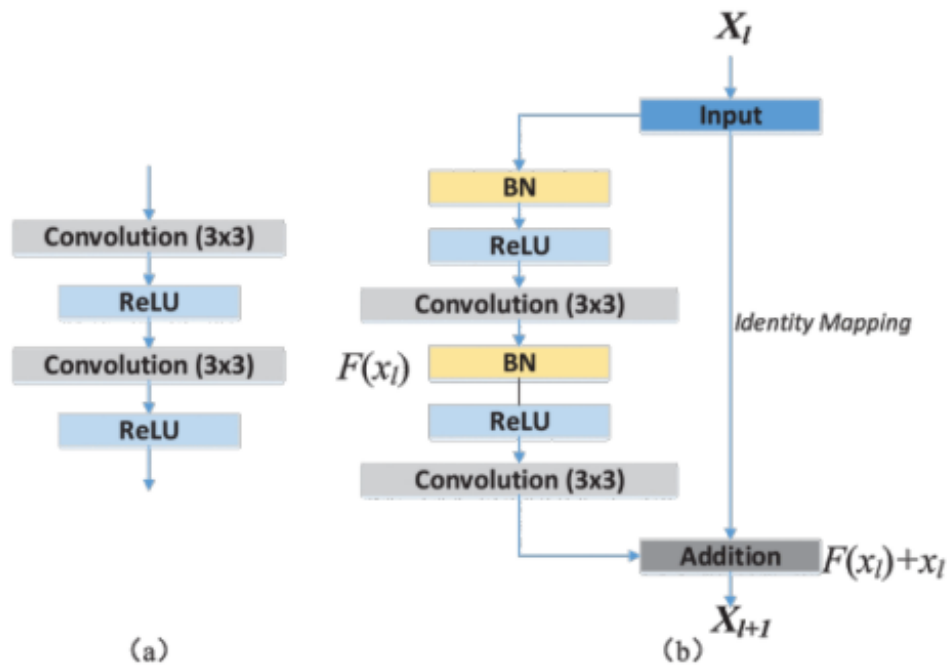


Figura 20: (a) Bloco padrão da UNet. (b) Bloco residual da ResUNet

1.3.4 DeepLab V1 & V2

A DeepLabV1 é uma arquitetura de rede neural convolucional projetada para segmentação semântica, que utiliza **atrous convolution** para aumentar o campo receptivo da rede sem aumentar o número de parâmetros. Ela também utiliza **image pooling** para trazer contexto global da imagem para a rede.

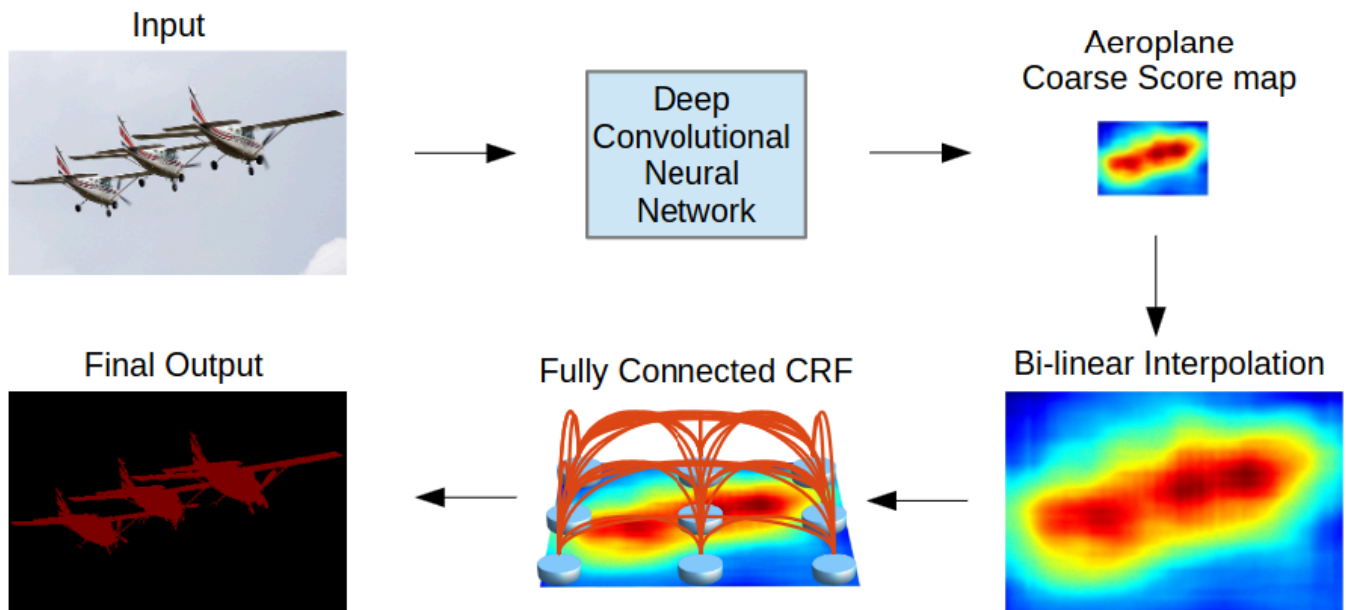


Figura 21: Arquitetura da DeepLabV1&V2

Ambas seguem uma arquitetura muito semelhante, diferindo por um único conceito. Na DeepLab V1, passamos a imagem por uma Deep Convolutional Neural Network (DCNN) para extrair features utilizando camadas de Atrous Convolution. Depois, pegamos o score map obtido e aplicamos um processo de **interpolação bilinear** para aumentar a dimensionalidade do score map, e por fim aplicamos o um algoritmo de pós-processamento chamado **Conditional Random Field (CRF)** para refinar a segmentação. Já na DeepLab V2, o processo é o mesmo, mas ao invés de aplicarmos

apenas uma Atrous Convolution, aplicamos múltiplas Atrous Convolutions com diferentes **rates** (ASPP).

1.3.5 PARSENet

Foi na ParseNet que surgiu a ideia do **image pooling**, gerando o contexto global da imagem para a rede, permitindo que ela utilize informações de diferentes níveis de abstração para melhorar a segmentação. Já vimos antes como esse conceito funciona, mas como ele é estruturado dentro da rede?

Primeiro a rede passa por uma rede convolucional padrão, depois o feature map gerado é passado por um **image pooling**, que gera um vetor de características que representa a imagem como um todo. Esse vetor é então redimensionado através de upsampling e concatenado com o feature map original, permitindo que a rede utilize informações de contexto global para melhorar a segmentação.

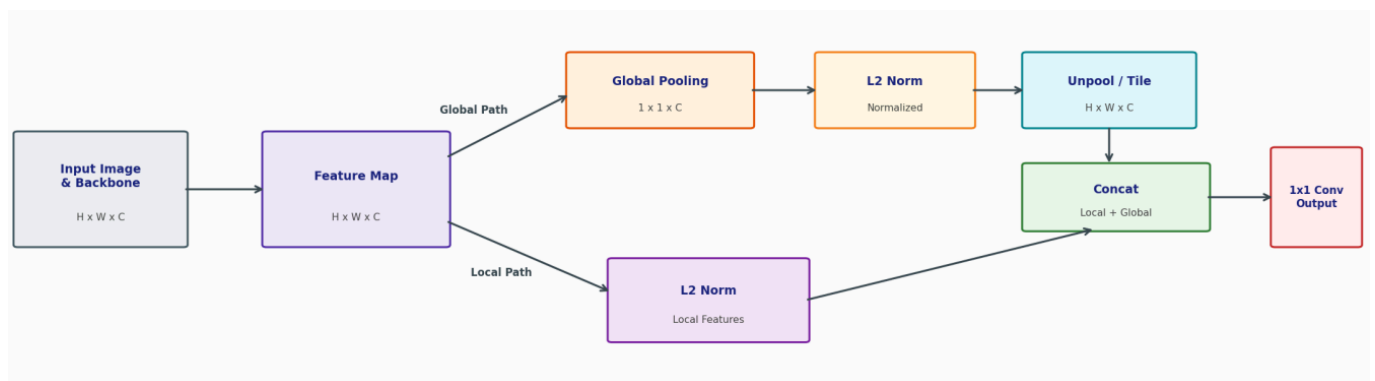


Figura 22: Arquitetura simplificada da PARSENet

1.3.6 PSPNet

A PSPNet é uma arquitetura de rede neural convolucional projetada para segmentação semântica, que utiliza **pyramid pooling** para capturar informações de diferentes escalas da imagem. Ela também utiliza **atrous convolution** para aumentar o campo receptivo da rede sem aumentar o número de parâmetros.

Primeiro a imagem passa por um processamento em uma CNN, essa rede diminui a imagem original para 1/8 do seu tamanho original e entra no bloco PSP **único**. Assim como no ASPP a imagem passa por múltiplas convoluções paralelas, no bloco PSP, a imagem passa por múltiplos **average pooling** com diferentes tamanhos de agrupamento:

- **Nível Red (1 × 1):** Realiza um Global Average Pooling sobre toda a extensão espacial. Gera um único vetor por canal que representa o contexto macro/global da cena inteira
- **Nível Orange (2 × 2):** Divide o mapa em 4 quadrantes (grade 2 × 2) e extrai a média de cada região (contexto regional amplo)
- **Nível Blue (3 × 3):** Divide o mapa em 9 sub-regiões (grade 3 × 3) para capturar um contexto regional médio
- **Nível Green (6 × 6):** Divide o mapa em 36 sub-regiões (grade 6 × 6) para capturar detalhes locais e regionais finos

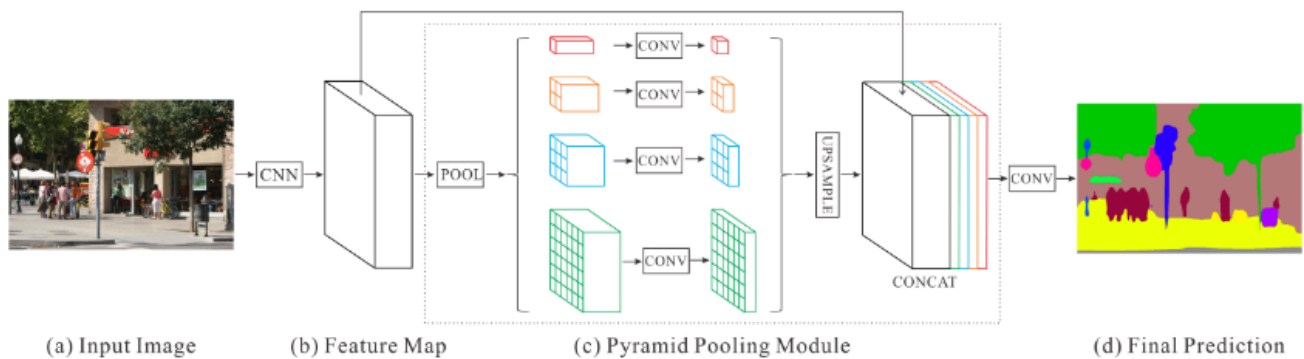


Figura 23: Arquitetura da PSPNet

então através do upsample, cada mapa de pooling é redimensionado para o tamanho original do feature map, e todos os mapas são concatenados, permitindo que a rede utilize informações de diferentes escalas para melhorar a segmentação.

1.3.7 Deeplab V3 & V3+

A DeeplabV3 teve algumas melhorias implementadas. O primeiro ponto foi a remoção do pós-processamento com CRF, que foi substituído por um **upsampling** simples, dessa forma a própria rede consegue aprender a mapear corretamente a segmentação. O segundo ponto foi a implementação do ASPP, que já havíamos comentado anteriormente. E o terceiro ponto foi a implementação, em paralelo com o ASPP, de um **image pooling** para pegar contexto global da rede

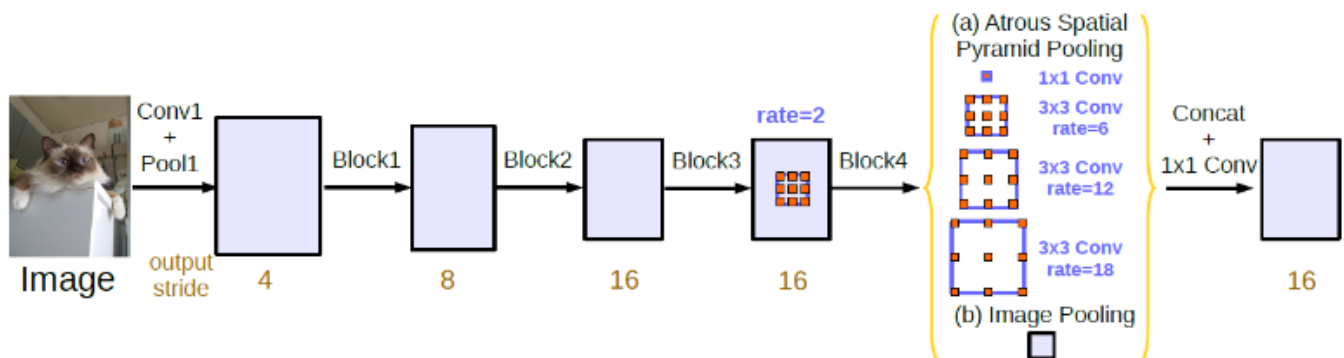


Figura 24: Arquitetura da DeeplabV3

O **DeepLabv3+** foi projetado para resolver uma limitação fundamental do DeepLabv3: embora o DeepLabv3 capturasse um contexto multi-escala excelente através do ASPP, ele perdia detalhes finos e precisão nas bordas dos objetos devido à redução de resolução espacial (**striding** e **pooling**) no backbone. Para corrigir isso, o DeepLabv3+ combina o melhor de duas abordagens: a extração de contexto do **Spatial Pyramid Pooling (ASPP)** com a capacidade de recuperação de bordas da estrutura **Encoder-Decoder**.

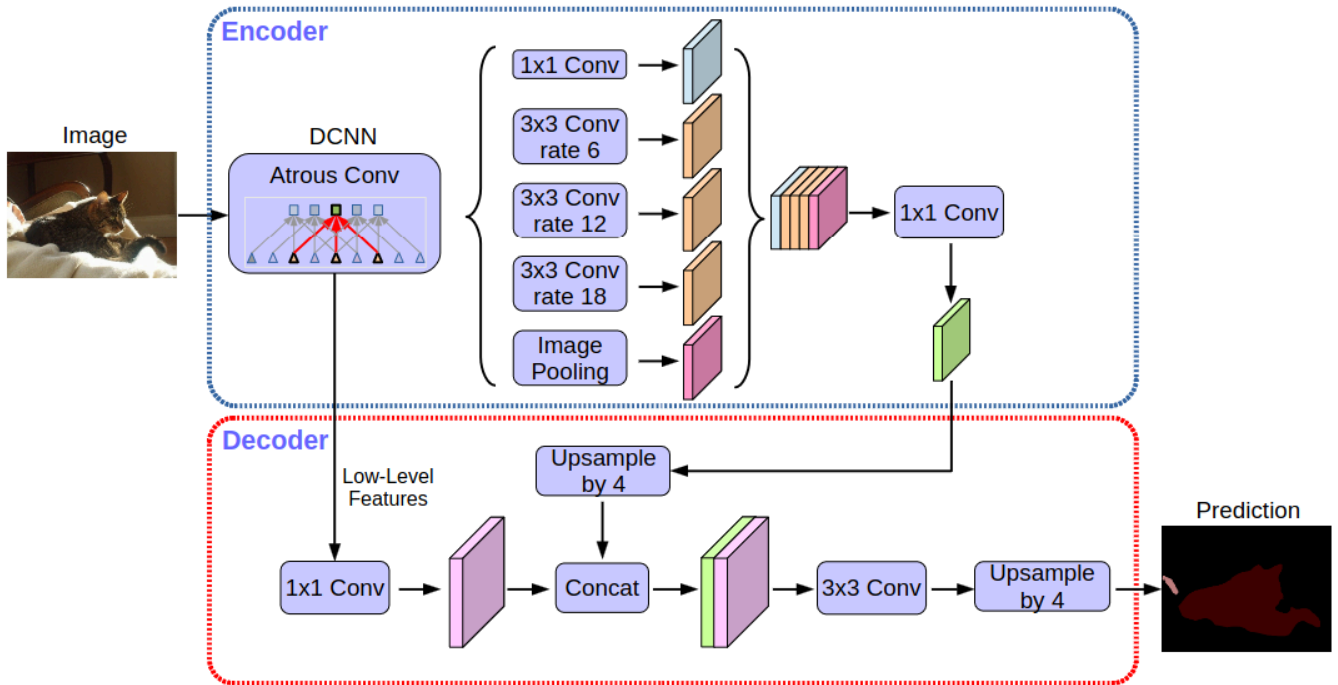


Figura 25: Arquitetura da DeepLabV3+

Em vez do upsampling bilinear direto que o V3 fazia, o V3+ agora tem um módulo decoder dedicado, onde ele faz upsampling das características e vai utilizando das features do encoder para refinar a segmentação, especialmente nas bordas dos objetos. Isso permite que o modelo mantenha a precisão espacial enquanto ainda aproveita o contexto global capturado pelo ASPP.

1.4 Percas

Agora vamos visualizar as losses que utilizamos para informar o modelo como aprender! Mas antes, vamos recapitular alguns pontos e definições.

Primeiramente, dado uma imagem $H \times W$, a saída dos nossos modelos deve ser uma matriz $H \times W$ onde cada elemento da matriz representa a classe do pixel correspondente na imagem (também pode ser um vetor com $N = H \cdot W$ elementos). Vamos considerar a abordagem de um único vetor de N elementos com a classe de cada pixel. Definiremos também t_i sendo a **classe verdadeira** do pixel i (ground truth). Definimos também p_i sendo a probabilidade que o modelo atribui para que i seja da sua **classe verdadeira** ($\mathbb{P}(\text{classe}(i) = t_i)$). A partir disso, podemos definir as losses que utilizamos para treinar nossos modelos.

1.4.1 Cross Entropy Loss

A primeira que vamos ver é a mais padrão para problemas de classificação, a **cross entropy loss**. Ela é definida como

$$\text{CE} = -\frac{1}{N} \sum_{i=1}^N \log(p_i) \quad (10)$$

se o modelo prevê alta probabilidade para a classe correta do pixel, o $\log(p_i)$ será próximo de 0, e a loss será pequena. Se o modelo prevê baixa probabilidade para a classe correta do pixel, o $\log(p_i)$ será negativo e a loss será grande. O objetivo do treinamento é minimizar essa loss, ajustando os pesos da rede para que ela preveja corretamente as classes dos pixels.

No entanto, essa loss carrega um problema. Quando existe um desbalanceamento de classes dentro do meu dataset, pode acontecer de a rede aprender a prever apenas a classe majoritária, ignorando as classes minoritárias.

1.4.2 Balanced Cross Entropy Loss

Para resolver o problema citado, podemos utilizar a **balanced cross entropy loss**, que atribui pesos diferentes para cada classe, penalizando mais os erros nas classes minoritárias.

$$\text{BCE} = -\frac{1}{N} \sum_{i=1}^N \omega_{t_i} \log(p_i) \quad (11)$$

o peso ω_{t_i} é pré-calculado de forma inversamente proporcional à frequência da classe t_i no dataset, de forma que classes minoritárias tenham pesos maiores e classes majoritárias tenham pesos menores. Isso força a rede a prestar mais atenção às classes minoritárias durante o treinamento. Podemos ter uma formulação binária também

$$\text{BCE} = -\frac{1}{N} \left(\sum_{i \in \text{positivos}} \omega_{\text{pos}} \log(p_i) + \sum_{i \in \text{negativos}} \omega_{\text{neg}} \log(p_i) \right) \quad (12)$$

1.4.3 Focal Loss

Essa loss serve para resolver um problema sutil. A loss anterior resolve o problema de desbalanceamento de classes, mas não resolve o problema de **hard examples**, ou seja, exemplos que são difíceis de classificar e quais são esses pixels? São justamente os que estão cada vez mais próximos das bordas do elemento. No entanto, os pixels fáceis costumam ser os mais presentes, e a soma do gradiente de sua contribuição pode atrapalhar no aprendizado dos pixels mais difíceis. A **focal loss** resolve esse problema, diminuindo a contribuição dos pixels fáceis para o gradiente, permitindo que a rede foque nos pixels mais difíceis.

$$\text{FL} = -\frac{1}{N} \sum_{i=1}^N (1 - p_i)^\gamma \log(p_i) \quad (13)$$

- **Comportamento do pixel fácil:** Consideremos $p_i = 0.95$ e $\gamma = 2$, então o fator de peso fica 0.0025, a perda desse pixel é reduzida em 99.75%, fazendo com que ele quase não afete o treinamento
- **Comportamento do pixel difícil:** Consideremos $p_i = 0.2$ e $\gamma = 2$, então o fator de peso fica 0.64, a perda desse pixel é mantida relevante

γ é o parâmetro focal, quanto maior ele é, mais severa é a penalidade aplicada aos pixels fáceis, e quanto menor ele é, mais leve é a penalidade aplicada aos pixels fáceis. O valor padrão de γ é 2, mas ele pode ser ajustado dependendo do problema e do dataset.

1.4.4 Balanced Focal Loss

Combina as duas soluções, ponderando cada pixel de acordo com sua classe e aplicando penalidade em pixels fáceis, de forma que a rede foque nos pixels mais difíceis e nas classes minoritárias.

$$\text{FL}_{\text{bal}} = -\frac{1}{N} \sum_{i=1}^N \omega_{t_i} (1 - p_i)^\gamma \log(p_i) \quad (14)$$

1.4.5 Loss Function for Regression

A saída por pixel pode não necessariamente ser um label de classe, mas um valor numérico. Por exemplo, se a rede estiver tentando estimar a profundidade aplicada àquela foto, então utilizamos as losses L_1 e L_2

$$L_1 = \frac{1}{N} \sum_{i=1}^N |y_i - t_i|$$

$$L_2 = \frac{1}{N} \sum_{i=1}^N (y_i - t_i)^2$$
(15)

onde y_i é o valor predito pelo modelo e t_i é o valor verdadeiro.

1.5 Pontos Práticos

1.5.1 A necessidade de Patches (Tiles)

Em aplicações reais, as imagens originais frequentemente possuem resoluções massivas (ex: 4000×4000 pixels ou mais). Tentar passar uma imagem desse tamanho inteira por uma rede neural de uma só vez estoura o limite de memória VRAM da GPU.

Por isso, na prática, a imagem é fatiada em blocos menores (**patches** ou **tiles**) para serem processados individualmente durante o treinamento e a inferência, montando-se um **mosaico** com os resultados finais de cada bloco.

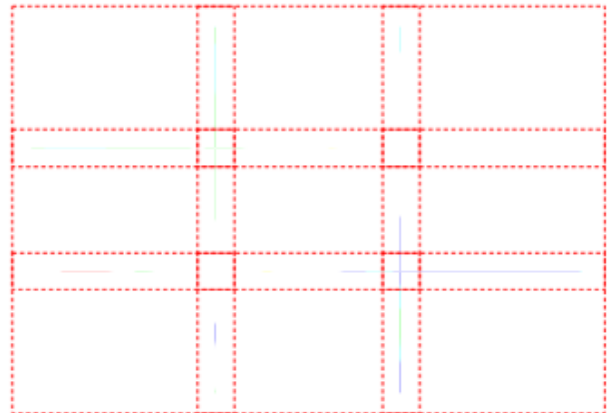


Figura 26: Exemplo de fatiamento de uma imagem em blocos menores

1.5.2 O Problema do Contexto nas Bordas (Border Context Loss)

Dividir a imagem em blocos cria um desafio geométrico nas extremidades de cada bloco:

- **Perda de Contexto Espacial:** Os pixels situados nas bordas de um **patch** perdem a vizinhança e o contexto espacial da região vizinha que ficou no **patch** ao lado.
- **Artefatos no Mosaico:** Ao colar os **patches** de volta para reconstruir a imagem final, essa falta de contexto nas margens gera artefatos de corte visíveis, descontinuidades e erros de classificação ao longo das linhas de junção dos blocos.

Para mitigar a perda de contexto nas bordas dos blocos, podemos destacar a estratégia de *mosaico com sobreposição* (**Overlapping Patches**):

1. **Sobreposição de Blocos:** Os **patches** são recortados com uma porcentagem de sobreposição em relação aos vizinhos, em vez de serem colados estritamente lado a lado.
2. **Considerar Apenas a Região Central (Inner Part):** Descarta-se a borda externa do **patch** (onde o contexto foi prejudicado) e utiliza-se apenas a previsão da região central válida.
3. **Média das Predições (Averaging Results):** Nas áreas onde os blocos se sobrepõem, calcula-se a média das probabilidades previstas por cada bloco para definir a classe final do pixel.

1.5.3 CNN Patch-wise Clássica vs. FCN Modela

- **CNN Clássica Por Patch (Sliding Window):** Classificava isoladamente o pixel central de um **patch** deslocado. Isso gerava alta redundância de cálculos e resultava em um efeito de **suavização excessiva nas bordas dos objetos (*oversmoothing*)**.
- **FCN Moderna:** Classifica todos os pixels do **patch** simultaneamente em uma única passada (**dense prediction**), aprendendo estruturas e geometrias específicas diretamente contidas dentro de cada bloco.

Object Detection

2.1 Introdução

Dentro da área de visão computacional, podemos fazer algumas distinções de tarefas, veja o gráfico abaixo

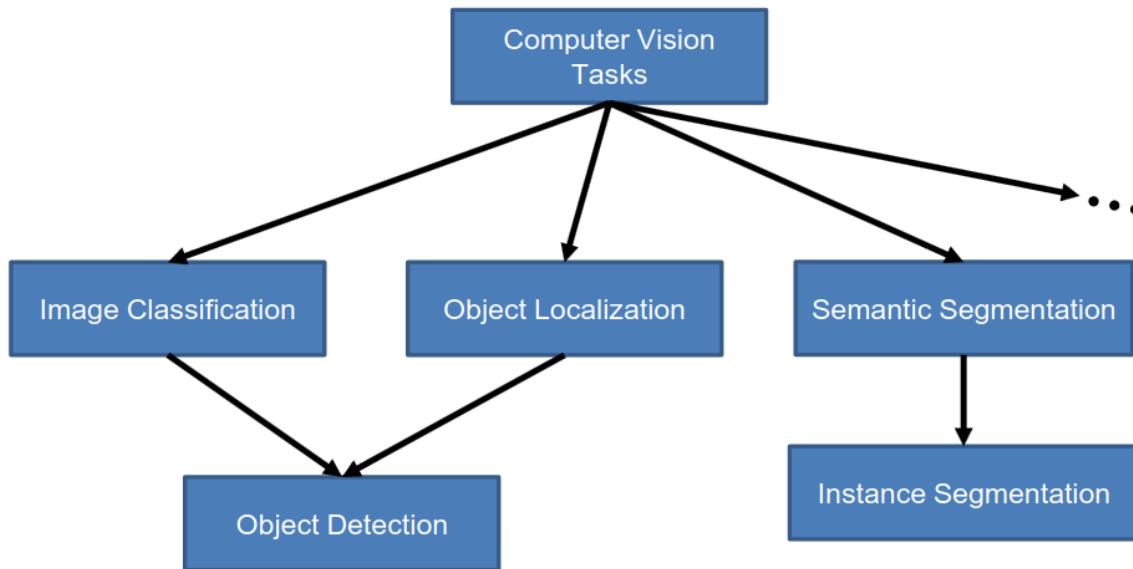


Figura 27: Áreas de visão computacional

Já vimos segmentação semântica, que é a tarefa de classificar cada pixel da imagem em uma categoria específica. A detecção de objetos, por outro lado, é a tarefa de identificar e localizar objetos específicos dentro de uma imagem, geralmente representados por **bounding boxes**. A detecção de objetos é muito baseada em **regiões** enquanto a segmentação semântica é baseada em **pixels**. A detecção de objetos é fundamental para diversas aplicações, como vigilância, direção autônoma e análise de imagens médicas.

2.2 Métricas de Avaliação

Podemos utilizar métricas já vistas como Definição 1.1.1.1, Definição 1.1.1.2 e Definição 1.1.1.3, porém podemos também utilizar métricas como **Recall**

Definição 2.2.1 (Recall):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (16)$$

A maioria das competições utiliza a **mean Average Precision (mAP)** como métrica principal, que é a média das precisões de cada classe, considerando diferentes limiares de confiança para as detecções. O mAP é derivado de valores *precision v.s recall*, fazendo uma variação do limiar de confiança para cada classe. O **limiar de confiança** é a probabilidade de que uma **caixa de âncora** contenha um objeto. Dado a Definição 1.1.1.3 de Average Precision, podemos definir melhor o mAP

Definição 2.2.2 (mAP): Dado que na minha imagem eu tenho mapeado K classes, a métrica de mean Average Precision (mAP) é definida como a média das precisões de cada classe:

$$\text{mAP} = \frac{1}{K} \sum_{k=1}^K \text{AP}_k \quad (17)$$

2.3 Redes de Estágio Único (Single-Shot): A Família YOLO

Baseada em redes neurais convolucionais que redefiniu o problema de detecção como uma única tarefa de regressão e classificação em uma só passada (**single-shot**).

Ao contrário das abordagens tradicionais baseadas em **sliding window** (janela deslizante), que executavam classificadores repetidamente sobre centenas de retalhos da imagem gerando um custo computacional altíssimo, o YOLO avalia a imagem inteira de uma só vez

2.3.1 Fundamentos do YOLO

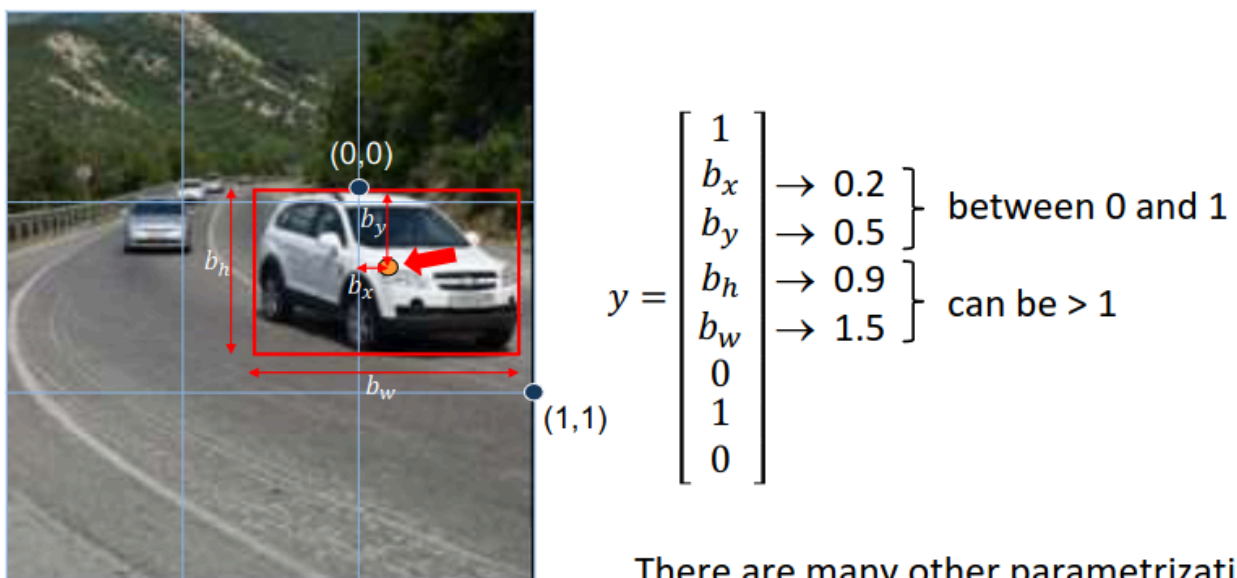
Um modelo YOLO (You Only Look Once) divide a imagem em uma grade de células, ele também recebe obrigatoriamente uma imagem quadrada, de lados $n \times n$. Cada célula da grade fica responsável por prever objetos cujo **ponto central** (mid point) caia dentro dos limites dessa célula. Como uma única passada na CNN processa todas as células simultaneamente, o YOLO é extremamente rápido e eficiente, tornando-o adequado para aplicações em tempo real.

2.3.2 Parametrização do vetor de saída

Para cada célula da grade, o YOLO prevê um vetor de saída que contém informações sobre os objetos detectados. Esse vetor inclui:

$$y = [p_{\text{obj}}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C] \quad (18)$$

- p_{obj} : Probabilidade de que a célula contenha um objeto
- (b_x, b_y, b_h, b_w) : Coordenadas da caixa delimitadora (x, y, largura, altura). $b_x, b_y \in [0, 1]$, representando a posição relativa do centro da caixa em relação à célula. Por exemplo, se $b_x = 0.5$ e $b_y = 0.5$, então a caixa de âncora está exatamente no centro da célula. b_h e b_w representam a altura e largura relativas à caixa, mas podem ser maiores que 1 (a caixa pode ser maior que a célula).



There are many other parametrizations.

Figura 28: Exemplo de caixa delimitadora prevista pelo YOLO

- $(c_1, c_2, \dots, c_C)$: Probabilidades de cada classe

2.3.3 Caixas de Ancoragem (Anchor Boxes)

Para resolver o problema de múltiplos objetos cujos centros caíam na mesma célula ou objetos de proporções muito distintas (como uma pessoa alta e um carro largo), o YOLO utiliza **Anchor Boxes**. Cada célula da grade prevê múltiplos **bounding boxes** associados a modelos geométricos

pré-definidos (**anchors**). Em vez de prever o formato absoluto da caixa do zero, a rede aprende deslocamentos (**offsets**) para ajustar a posição e a dimensão das **anchor boxes pré-definidas**

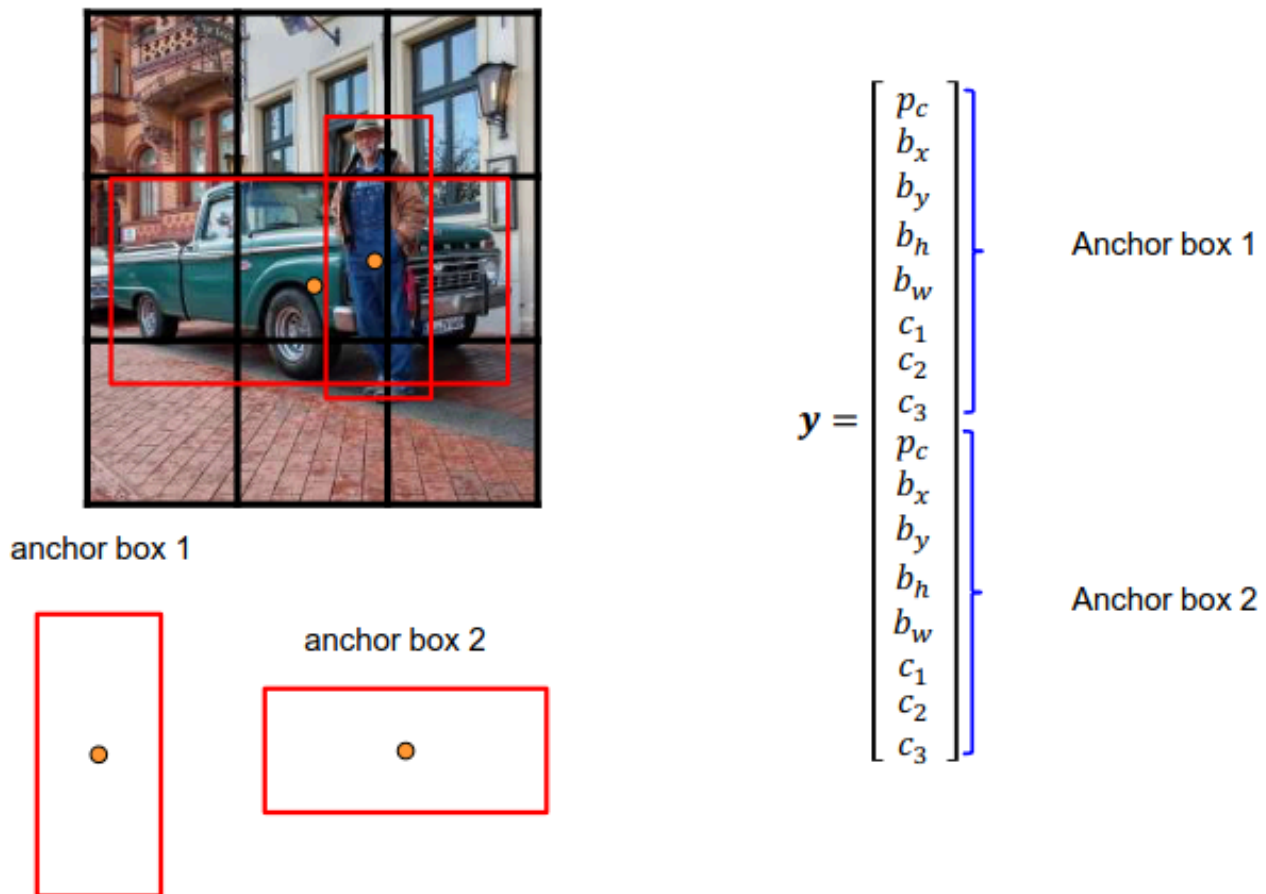


Figura 29: Exemplo de caixas de ancoragem previstas pelo YOLO

Então nessa nova formulação, o vetor de saída para cada célula da grade se torna:

$$y = \begin{pmatrix} [p_{\text{obj}}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C]_{\text{anchor 1}} \\ [p_{\text{obj}}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C]_{\text{anchor 2}} \\ \vdots \\ [p_{\text{obj}}, b_x, b_y, b_h, b_w, c_1, c_2, \dots, c_C]_{\text{anchor A}} \end{pmatrix} \quad (19)$$

Vale ressaltar que eu escrevi em forma de matriz, no entanto o mais comum é um vetor contínuo e separamos as anchor boxes pelo padrão da saída, que seria a cada $5 + C$ valores, onde C é o número de classes. Por exemplo, se temos 3 anchor boxes e 20 classes, o vetor de saída para cada célula da grade terá tamanho $3 \times (5 + 20) = 75$.

2.3.4 Supressão Não-Máxima (Non-Maximum Suppression - NMS)

Na prática, é bem fácil perceber que vai acontecer de várias caixas serem selecionadas para o mesmo objeto, e isso é um problema. Para resolver isso, utilizamos a técnica para escolher a caixa que melhor representa o objeto, descartando as demais. A técnica é chamada de **Non-Maximum Suppression (NMS)**, e funciona da seguinte forma:

1. **Filtragem por confiança:** Descartamos todas as caixas cuja probabilidade de conter um objeto seja menor que um limiar pré-definido (ex: 0.5)
2. **Seleção da melhor caixa:** Entre as caixas restantes, selecionamos a caixa com a maior probabilidade de conter um objeto (maior pontuação de confiança p_{obj})

3. **Eliminação de duplicatas:** Elimina as outras caixas da mesma classe que possuem uma sobreposição $\text{IoU} \geq 0.5$ com a caixa selecionada
4. Repete o processo iterativamente para as caixas restantes de cada classe

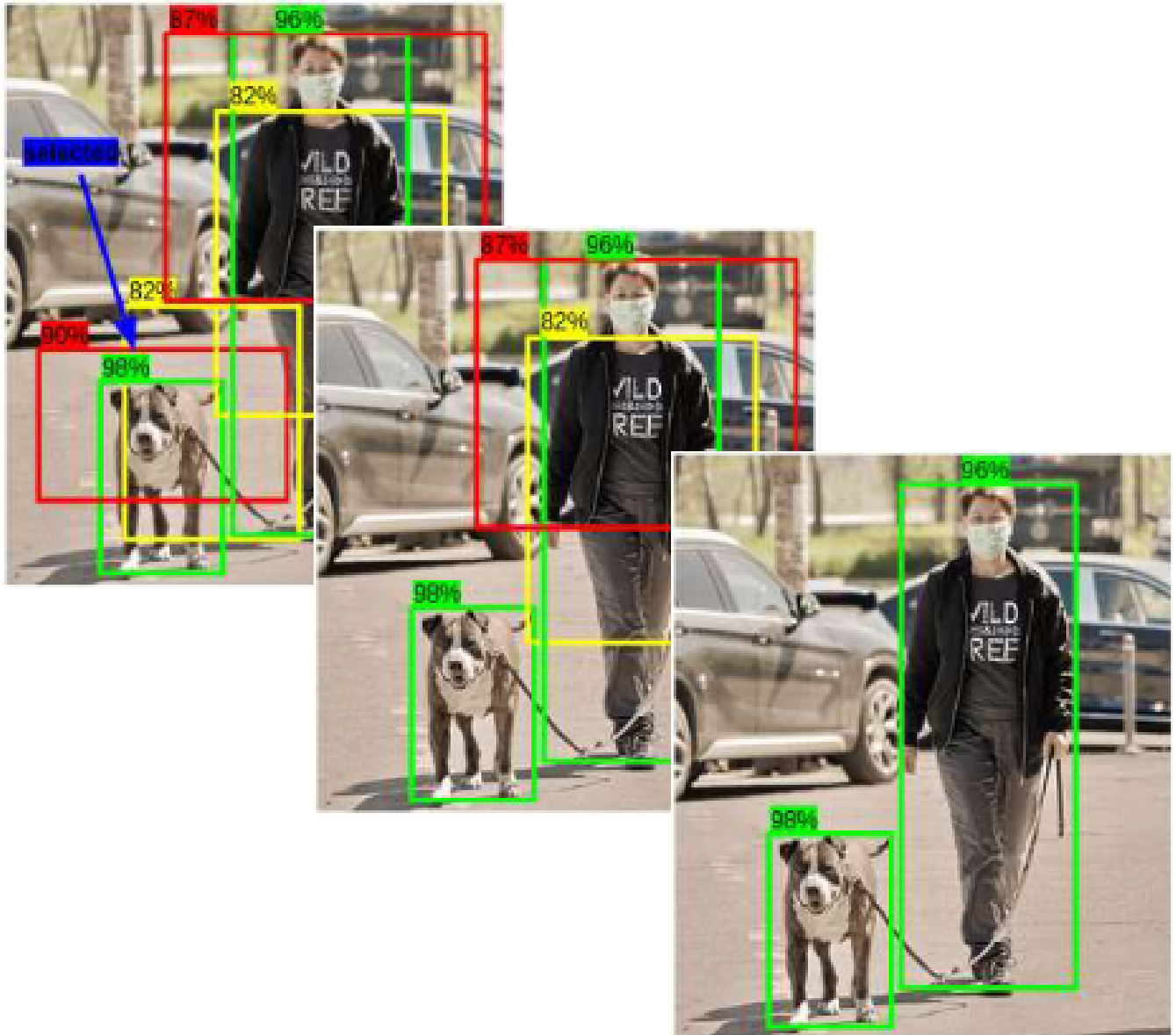


Figura 30: Exemplo de supressão não-máxima

Vale ressaltar que esse **pós-processamento** é feito com **todas** as caixas de ancoragem previstas, tanto as que foram atribuídas dentro de uma mesma célula (uma única célula atribui diferentes anchor boxes pro mesmo objeto) quanto as geradas por células vizinhas (várias células podem prever o mesmo objeto). O objetivo é garantir que cada objeto seja representado por uma única caixa delimitadora final.

2.3.5 Evolução Arquitetural

2.3.5.1 YOLOv1 & YOLO9000

Arquiteturas iniciais, foi na YOLO9000 onde as anchor boxes foram introduzidas e, em vez de prever diretamente posição e tamanho das caixas, a rede aprende a prever **offsets** para ajustar as **anchor boxes** pré-definidas.

2.3.5.2 YOLOv3

Aumento da profundidade da rede, de 53 camadas para 106 camadas. Adição de skip connections para melhorar a propagação do gradiente e permitir que a rede aprenda representações mais complexas. Introdução de **multi-scale predictions**, onde a rede prevê caixas em três escalas diferentes, permitindo detectar objetos de tamanhos variados.

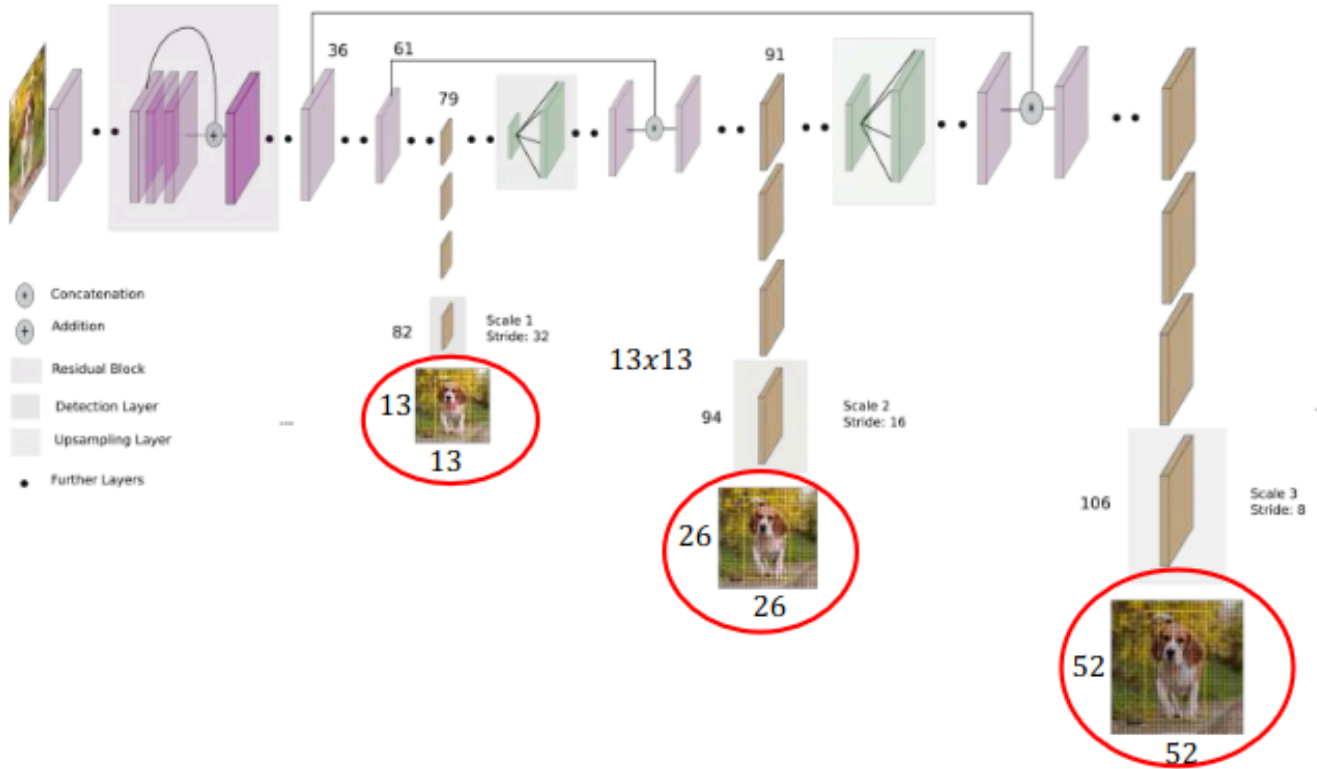


Figura 31: Arquitetura da YOLOv3

2.3.6 Loss Function

Não podemos falar de um modelo de rede sem discutir a loss por ela utilizada. A loss do YOLO é composta por três partes principais: a **loss de localização**, a **loss de confiança** e a **loss de classificação**. A **loss de localização** mede o quão bem a rede prevê as coordenadas da caixa delimitadora em relação à caixa real. A **loss de confiança** avalia a precisão da rede em prever se uma caixa contém um objeto ou não. A **loss de classificação** mede a precisão da rede em classificar corretamente o objeto dentro da caixa.

Vamos rapidamente definir que a **predição do modelo** é dada, para a célula $i \in \{1, \dots, S^2\}$ e anchor box $j \in \{1, \dots, B\}$:

$$y_{ij} = [p_{\text{obj}}^{(ij)}, b_x^{(ij)}, b_y^{(ij)}, b_h^{(ij)}, b_w^{(ij)}, c_1^{(ij)}, c_2^{(ij)}, \dots, c_C^{(ij)}] \quad (20)$$

e vamos considerar o vetor **ground truth** como

$$t_{ij} = [t_0^{(ij)}, t_x^{(ij)}, t_y^{(ij)}, t_h^{(ij)}, t_w^{(ij)}, s_1^{(ij)}, s_2^{(ij)}, \dots, s_C^{(ij)}] \quad (21)$$

então a loss da YOLO é dada por

$$\begin{aligned}
\mathcal{L}_{\text{YOLO}} = & \underbrace{\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^B t_0^{(ij)} \left((t_x^{(ij)} - b_x^{(ij)})^2 + (t_y^{(ij)} - b_y^{(ij)})^2 + (t_h^{(ij)} - b_h^{(ij)})^2 + (t_w^{(ij)} - b_w^{(ij)})^2 \right)}_{\text{Loss de localização}} \\
& + \underbrace{\lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^B (1 - t_0^{(ij)}) \left(-\log(1 - p_{\text{obj}}^{(ij)}) \right)}_{\text{Loss de confiança}} \\
& + \underbrace{\sum_{i=0}^{S^2} \sum_{j=0}^B t_0^{(ij)} \left[-\log(p_{\text{obj}}^{(ij)}) + \sum_{k=1}^C \text{BCE}(c_k^{(ij)}, s_k^{(ij)}) \right]}_{\text{Loss de classificação}}
\end{aligned} \tag{22}$$

Onde S é a proporção que dividimos os grids da imagem, B é o número de **anchor boxes** por célula, C é o número de classes, λ_{noobj} e λ_{coord} são hiperparâmetros que controlam a importância relativa das diferentes partes da loss.

Vejamos como a loss se comporta. Se uma anchor box **não possui objeto**, então apenas a parte da **loss de confiança** é computada. Quando a probabilidade de que a caixa contenha um objeto é 0, então a $\log(1 - 0) = 0$, não penalizando a rede, no entanto, se a rede classificou como existindo um objeto, então a loss será alta e irá penalizar a rede. Se uma anchor box **possui objeto**, então apenas as partes da **Loss de Localização** e **Loss de Classificação** são computadas. A **Loss de Localização** mede o quão bem a rede prevê as coordenadas da caixa delimitadora em relação à caixa real, e a **Loss de Classificação** mede a precisão da rede em classificar corretamente o objeto dentro da caixa.

2.4 Redes de Dois Estágios e Segmentação de Instâncias: Mask R-CNN

As redes Mask R-CNN são uma extensão das redes Faster R-CNN, projetadas para realizar não apenas a detecção de objetos, mas também a segmentação de instâncias. A Mask R-CNN adiciona um ramo adicional à arquitetura Faster R-CNN para prever máscaras binárias para cada objeto detectado, permitindo que a rede identifique não apenas a localização e a classe do objeto, mas também sua forma precisa.

2.4.1 Geração de propostas com a RPN (Region Proposal Network)

Antes de mais nada, a **imagem é redimensionada** para caber na rede backbone. Essa rede backbone recebe a imagem e gera um **feature map** que representa as características da imagem. Esse **feature map** é então passado para a **Region Proposal Network (RPN)**, que é responsável por gerar propostas de regiões onde objetos podem estar localizados. A RPN utiliza um conjunto de k **anchor boxes** de diferentes tamanhos e proporções para cobrir uma variedade de objetos possíveis na imagem.

A RPN fica responsável por aprender principalmente duas coisas: A probabilidade de a anchor box conter ou não um objeto e estimar o tamanho/formato da caixa delimitadora do objeto. Primeiro, uma convolução 3×3 com 512 filtros é aplicada ao **feature map** da backbone, gerando um **feature map** intermediário.

Em seguida, duas convoluções 1×1 são aplicadas: uma para prever a probabilidade de cada **anchor box** conter um objeto (**objectness score**), contento um total de 36 filtros e outra para prever os ajustes necessários para refinar as coordenadas da caixa delimitadora (**bounding box regression**) com 18 filtros.

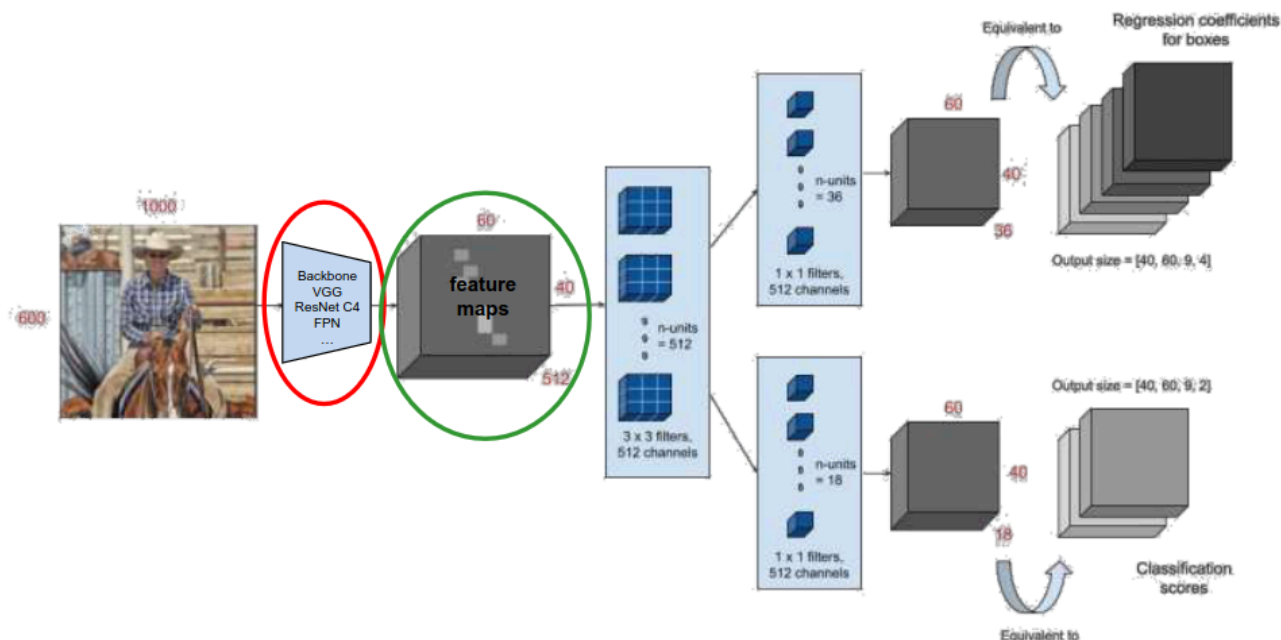


Figura 32: Arquitetura da Mask R-CNN

Logo após isso, para escolher as melhores propostas de regiões, a RPN aplica a técnica de **Non-Maximum Suppression (NMS)** para eliminar propostas redundantes e manter apenas as mais promissoras. As propostas selecionadas são então passadas para a próxima etapa da Mask R-CNN, onde cada proposta é processada individualmente para prever a classe do objeto, refinar a caixa delimitadora e gerar a máscara binária correspondente.

2.4.2 Alinhamento de Características com RoIAlign

Temos um problema, o próximo passo (a rede geradora de máscara) espera um **feature map** de tamanho fixo, mas as propostas de regiões geradas pela RPN podem ter tamanhos variados. Para resolver isso, a Mask R-CNN utiliza o **RoIAlign**, que é uma técnica que extrai características de regiões de interesse (RoIs) do **feature map** da backbone, garantindo que cada RoI seja representada por um **feature map** de tamanho fixo.

A primeira abordagem utilizada era a **RoIPooling**, que dividia a região de interesse em uma grade de células e aplicava **max pooling** em cada célula para obter um valor representativo. No exemplo da Figura 33, temos uma imagem de tamanho 8×8 e a região vermelha destacada é a região de interesse com tamanho 6×4 , no entanto a rede geradora de máscara espera uma **feature map** de tamanho 2×2 , então dividimos a região de interesse em uma grade de 2×2 células, e aplicamos **max pooling** em cada célula para obter um valor representativo. O problema é que a divisão da região de interesse em células pode não ser exata, resultando em perda de informações e desajustes na localização das características.

Veja por exemplo a Figura 34. Nesse caso, a região de interesse (borda vermelha) não cai exatamente na divisão dos pixels, na verdade ela para na metade de um, e isso **pode acontecer** como já discutimos anteriormente. Nesses casos, o **RoIPooling** arredonda os valores para o inteiro mais próximo (borda azul), resultando em perda de informações e desajustes na localização das características. Para resolver esse problema, a Mask R-CNN utiliza o **RoIAlign**, que utiliza interpolação bilinear para calcular os valores das células da grade, garantindo que as características sejam alinhadas corretamente com a região de interesse.

0.1	0.3	0.2	0.3	0.2	0.6	0.8	0.9
0.4	0.5	0.1	0.4	0.7	0.1	0.4	0.3
0.2	0.1	0.3	0.8	0.6	0.2	0.1	0.1
0.4	0.6	0.2	0.1	0.3	0.6	0.1	0.2
0.1	0.8	0.3	0.3	0.5	0.3	0.3	0.3
0.2	0.9	0.4	0.5	0.1	0.1	0.1	0.2
0.3	0.1	0.8	0.6	0.3	0.3	0.6	0.5
0.5	0.5	0.2	0.1	0.1	0.2	0.1	0.2

0.8	0.7
0.8	0.6

Figura 33: Exemplo de RoIPooling com região de interesse alinhada com a grade de células

0.1	0.3	0.2	0.3	0.2	0.6	0.8	0.9
0.4	0.5	0.1	0.4	0.7	0.1	0.4	0.3
0.2	0.1	0.3	0.8	0.6	0.2	0.1	0.1
0.4	0.6	0.2	0.1	0.3	0.6	0.1	0.2
0.1	0.8	0.3	0.3	0.5	0.3	0.3	0.3
0.2	0.9	0.4	0.5	0.1	0.1	0.1	0.2
0.3	0.1	0.8	0.6	0.3	0.3	0.6	0.5
0.5	0.5	0.2	0.1	0.1	0.2	0.1	0.2

0.8	0.7
0.8	0.6

Figura 34: Exemplo de RoIPooling com região de interesse desalinhada com a grade de células

0.1	0.3	0.2	0.3	0.2	0.6	0.8	0.9
0.4	0.5	0.1	0.4	0.7	0.1	0.4	0.3
0.2	0.1	0.3	0.8	0.6	0.2	0.1	0.1
0.4	0.6	0.2	0.1	0.3	0.6	0.1	0.2
0.1	0.8	0.3	0.3	0.5	0.3	0.3	0.3
0.2	0.9	0.4	0.5	0.1	0.1	0.1	0.2
0.3	0.1	0.8	0.6	0.3	0.3	0.6	0.5
0.5	0.5	0.2	0.1	0.1	0.2	0.1	0.2

0.81	0.56
0.73	0.43

Figura 35: Exemplo de RoIAlign com região de interesse alinhada com a grade de células

2.4.3 Predições em Paralelo (R-CNN + FCN)

Depois que o **RoIAlign** é aplicado, cada proposta de região é representada por um **feature map** de tamanho fixo. Esse **feature map** é então passado para dois ramos **paralelos** da Mask R-CNN:

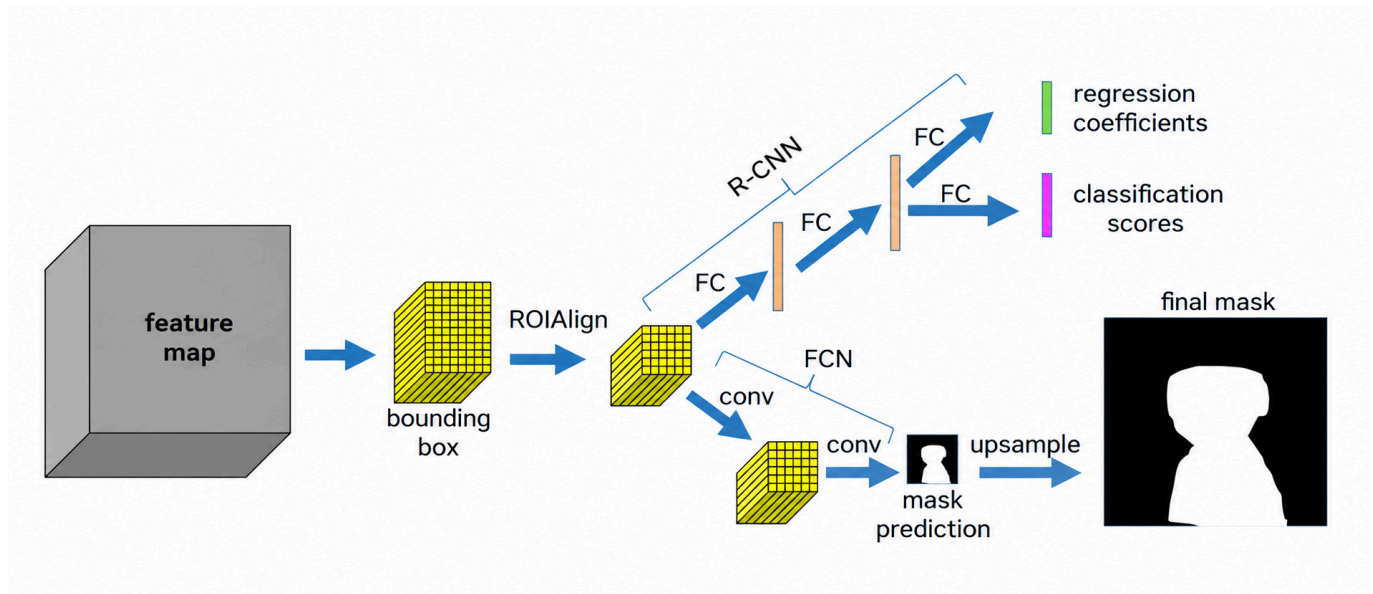


Figura 36: Arquitetura da Mask R-CNN com predições em paralelo

- **R-CNN**: Responsável por prever a classe do objeto e refinar a caixa delimitadora. Ele utiliza uma série de camadas totalmente conectadas para processar o **feature map** da proposta de região e gerar as predições de classe e caixa.
- **FCN**: Responsável por gerar a máscara binária do objeto. Ele utiliza uma série de camadas convolucionais para processar o **feature map** da proposta de região e gerar a máscara binária correspondente. A saída do FCN é uma máscara de tamanho fixo (por exemplo, 28×28 pixels) que representa a forma do objeto dentro da caixa delimitadora. Essa máscara é então redimensionada para se ajustar à caixa delimitadora refinada, permitindo que a Mask R-CNN produza uma segmentação precisa da instância do objeto.

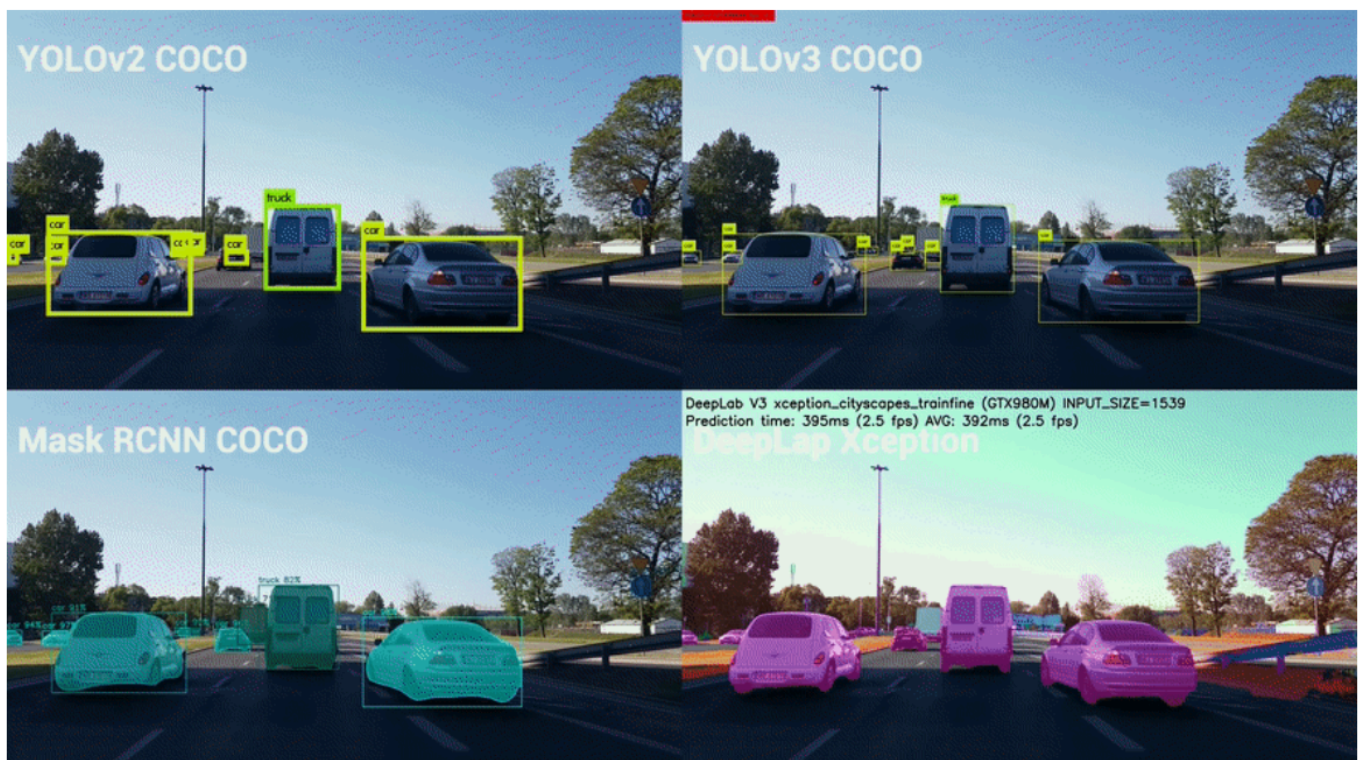


Figura 37: Arquitetura da Mask R-CNN com predições em paralelo

Recurrent Neural Networks (RNNs)

3.1 Introdução e Modelagem de Dados Sequenciais

As RNNs são redes neurais projetadas para lidar com dados sequenciais, onde os dados possuem um tipo de relação sequencial, seja por tempo, posição ou qualquer outra forma de dependência entre os elementos da sequência. Diferente das redes feedforward tradicionais, as RNNs possuem conexões recorrentes que permitem que informações de etapas anteriores da sequência influenciem a saída atual, tornando-as ideais para tarefas como processamento de linguagem natural, reconhecimento de fala, séries temporais e processamento de vídeos.

3.2 Simple RNN (Vanilla RNN)

É o modelo mais básico de RNN, onde cada célula da rede recebe a entrada atual e o estado oculto da etapa anterior, processando essas informações para gerar uma saída e atualizar o estado oculto para a próxima etapa da sequência.

Antes de partir para a estrutura matemática em si, vamos ver esse simples exemplo: Imagine uma cafeteria, e a Figura 38 representa a sequência de pratos principais que um cliente pode pedir ao longo de uma semana. Cada dia da semana representa uma etapa da sequência. A RNN é capaz de capturar essa dependência sequencial, permitindo que a rede aprenda padrões de pedidos ao longo do tempo.

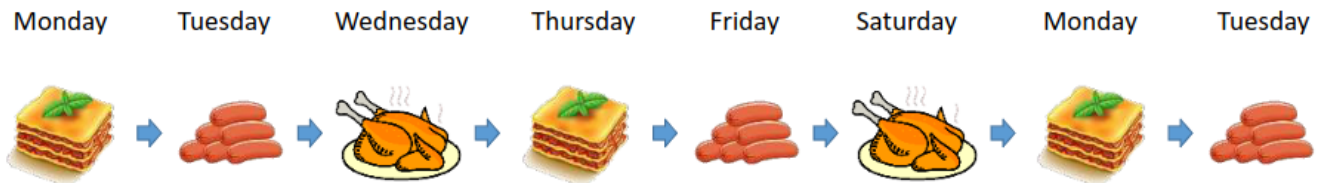


Figura 38: Exemplo de RNN em uma cafeteria

Podemos interpretar cada um dos pratos principais como vetores **one-hot**

$$\text{Lasanha} = [1, 0, 0] \quad \text{Salsicha} = [0, 1, 0] \quad \text{Frango} = [0, 0, 1] \quad (23)$$

E percebe que a seguinte relação é apresentada

$$\text{Lasanha} \rightarrow \text{Salsicha} \rightarrow \text{Frango} \rightarrow \text{Lasanha} \rightarrow \text{Salsicha} \rightarrow \text{Frango} \rightarrow \text{Lasanha} \quad (24)$$

Conseguimos facilmente representar essa rotatividade utilizando de uma matriz de transição:

$$X = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \quad (25)$$

$$X \text{ Lasanha} = \text{Salsicha} \quad X \text{ Salsicha} = \text{Frango} \quad X \text{ Frango} = \text{Lasanha}$$

Considere agora a influência de uma variável externa na decisão do restaurante com relação ao prato do dia, digamos o **clima**. Agora além dos vetores de pratos principais, temos também vetores **one-hot** representando o clima:

$$\text{Sol} = [1, 0] \quad \text{Chuva} = [0, 1] \quad (26)$$

Agora a rede depende tanto das informações de **histórico** (pratos anteriores) quanto das informações de **contexto** (clima atual). A rede agora aprende a prever isso através de **duas** matrizes

- W_{hh} : Matriz que aprende as regras da sequência do cardápio, ela responde a pergunta “Se ontem foi servido Frango, qual seria a mesma comida e qual será a próxima?”

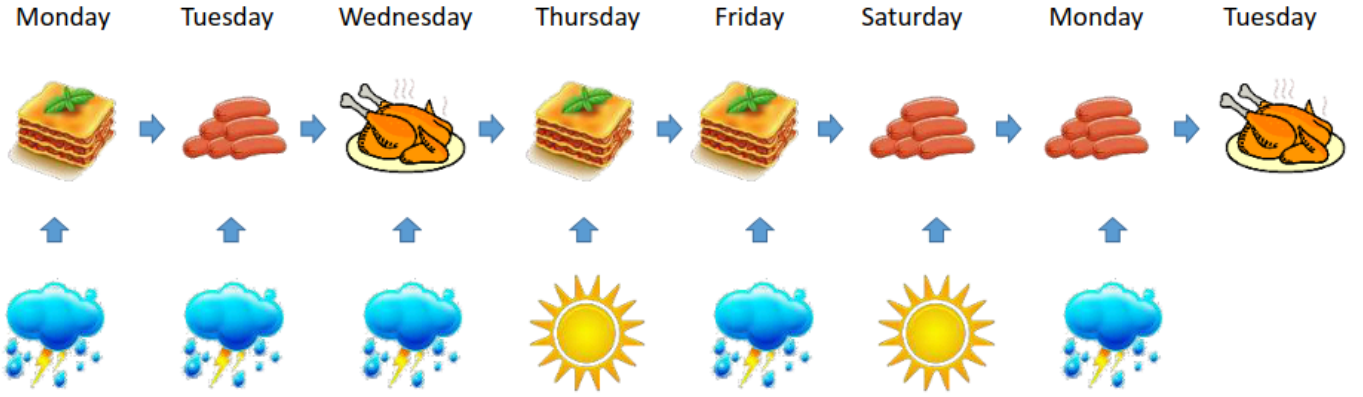


Figura 39: Exemplo de RNN em uma cafeteria com influência do clima

- W_{xh} : Matriz que aprende a influência do clima na decisão do cardápio, ela responde a pergunta “Se hoje está chovendo, devo manter a escolha de comida ou mudar para a próxima da sequência?”

Mas como a rede toma a decisão? A cada passo temporal (dia t), a rede utiliza a seguinte fusão: **Consulta a memória** pegando a comida do dia anterior h_{t-1} e multiplica por W_{hh} para entender a tendência do cardápio, **lê o presente** pegando o clima atual x_t e multiplica pela matriz W_{xh} , depois **soma e aplica ativação** e **gera a saída**, o novo estado oculto h_t que passa por uma matriz final W_{ho} que decide qual será a comida do dia t .

Definindo de forma mais formal, a RNN pode ser definida pela seguinte estrutura

$$h_t = f_{\theta}(h_{t-1}, x_t) = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h) \quad (27)$$

e a saída no instante t é dada por

$$o_t = W_{ho}h_t + b_o \quad (28)$$

Podemos representar toda essa estruturação em bloco da seguinte forma

$$h_t = \text{NL} \left([W_{hh}|W_{xh}|b_h] \begin{pmatrix} h_{t-1} \\ x_t \\ 1 \end{pmatrix} \right) \quad (29)$$

$$o_t = [W_{ho}|b_o] \begin{pmatrix} h_t \\ 1 \end{pmatrix}$$

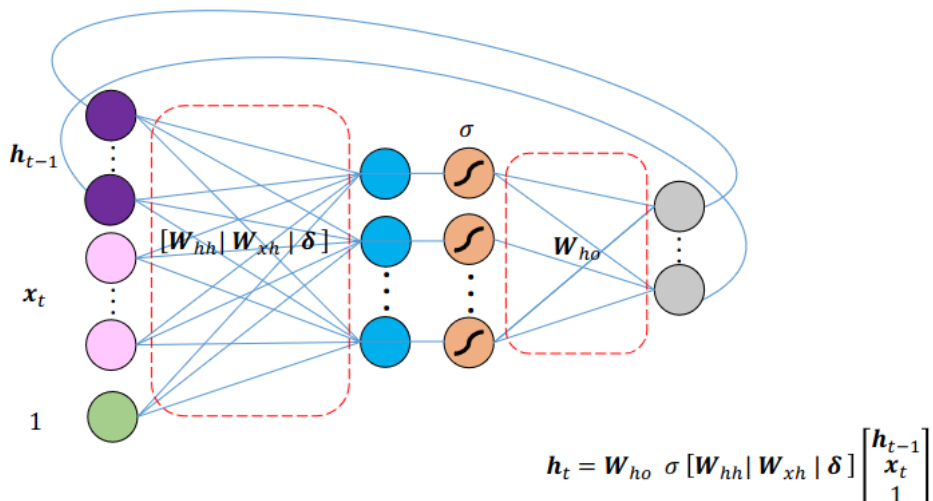


Figura 40: Bloco de uma RNN

3.2.1 Arquitetura

Podemos estruturar uma arquitetura visual fixa para cada um dos passos temporais que a rede recorrente faz

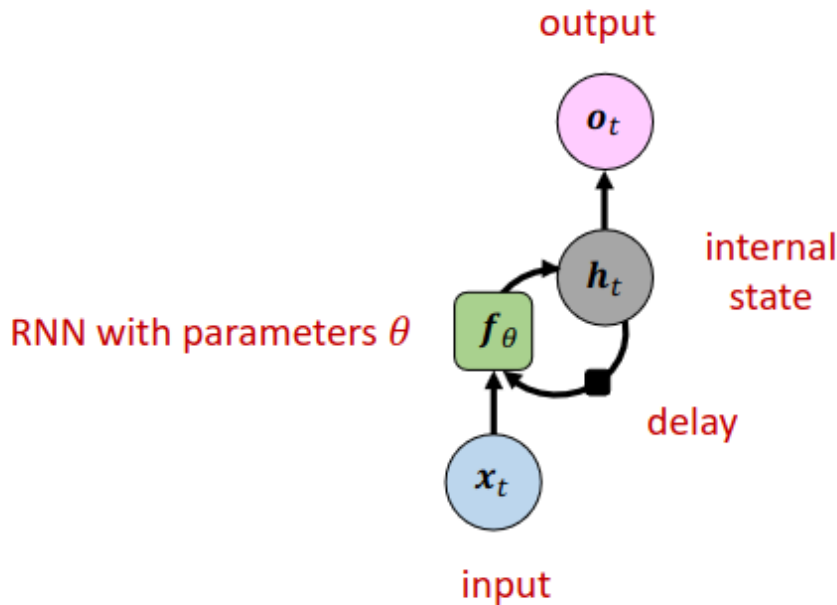


Figura 41: Arquitetura de uma RNN

Podemos utilizar uma fórmula **recursiva** para representar a RNN, onde o estado oculto h_t é atualizado a cada passo temporal com base no estado oculto anterior h_{t-1} e na entrada atual x_t .

$$\underbrace{h_t}_{\text{Estado Oculto}} = f_{\theta} \left(\underbrace{h_{t-1}}_{\text{Estado Anterior}}, \underbrace{x_t}_{\text{Entrada Atual}} \right) \quad (30)$$

e vale ressaltar que o **mesmo conjunto de parâmetros** θ é utilizado em **todos os passos temporais**, diferente de uma rede neural padrão onde **cada camada possui um conjunto de parâmetros diferente**. Isso permite que a RNN generalize melhor para sequências de diferentes comprimentos e capture padrões temporais de forma mais eficiente.

Tomemos a simples RNN $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$ e vamos ver como a recursividade se comporta com $T = 3$

$$\begin{aligned} h_3 &= \tanh(W_{hh}h_2 + W_{xh}x_3) \\ h_3 &= \tanh(W_{hh}(\tanh(W_{hh}h_1 + W_{xh}x_2)) + W_{xh}x_3) \\ h_3 &= \tanh(W_{hh}(\tanh(W_{hh}(\tanh(W_{hh}h_0 + W_{xh}x_1)) + W_{xh}x_2)) + W_{xh}x_3) \end{aligned} \quad (31)$$

3.2.2 RNN Unrolling

Baseado na arquitetura mostrada, podemos escolher que o output da rede seja o **output** de cada passo temporal, ou apenas o **output** do último passo temporal. A primeira abordagem é útil quando queremos prever uma sequência de saídas, enquanto a segunda abordagem é útil quando queremos prever uma única saída baseada em toda a sequência de entradas.

Baseado nisso, conseguimos desenvolver o parâmetro de tempo da RNN, mostrando como a rede processa cada elemento da sequência ao longo do tempo. Esse processo é conhecido como **unrolling** da RNN, e nos permite visualizar claramente como as informações fluem através da rede em cada passo temporal.

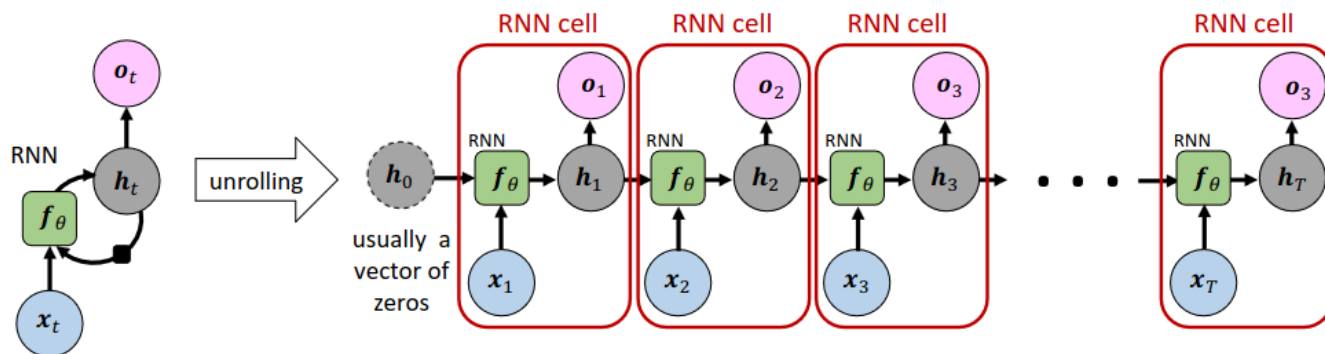


Figura 42: Desenrolando uma RNN

3.2.3 Tipos de mapeamento sequencial

- **Many-to-many:** Existem duas variações, a primeira é quando a entrada e a saída são sequências de comprimentos iguais. Por exemplo, os momentos de um vídeo e a categoria que aquele momento se encaixa (drama, terror, etc.)

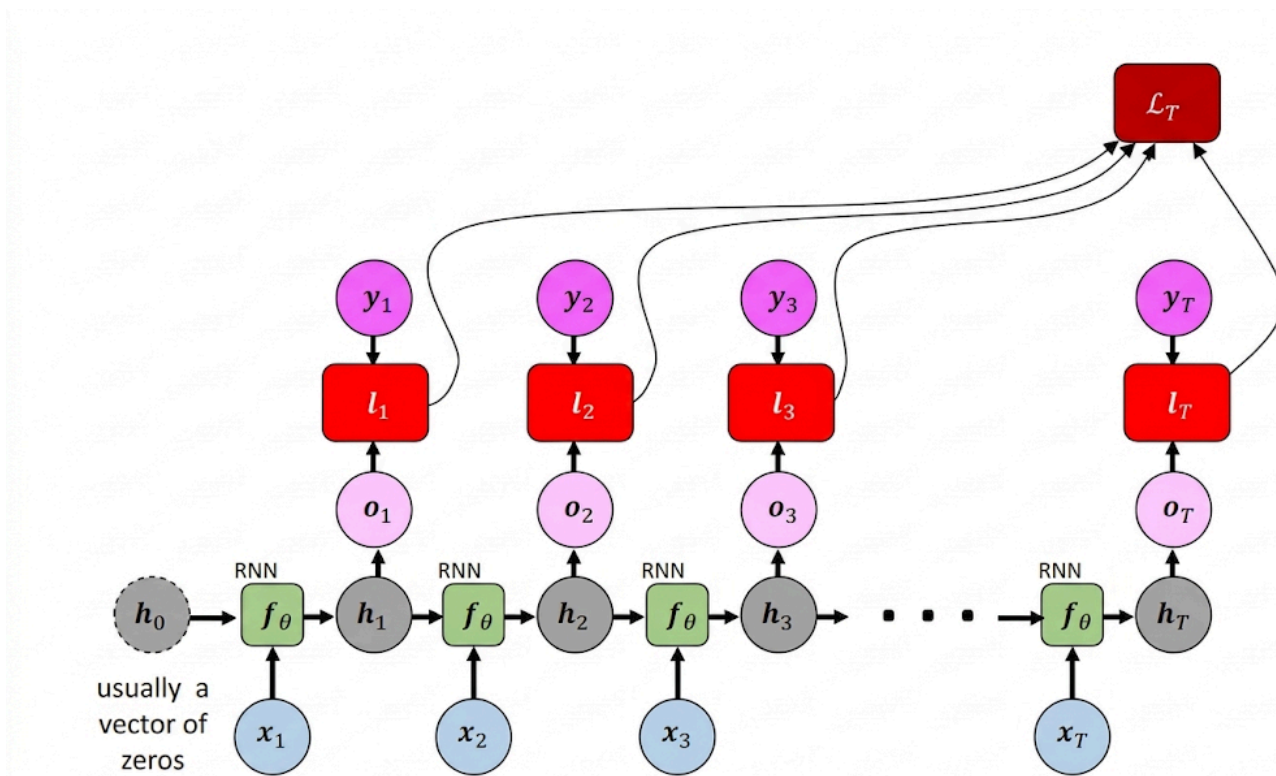


Figura 43: Exemplo de mapeamento many-to-many

A segunda variação é quando a entrada e a saída são sequências de comprimentos diferentes. Por exemplo, uma frase em inglês e sua tradução em português.

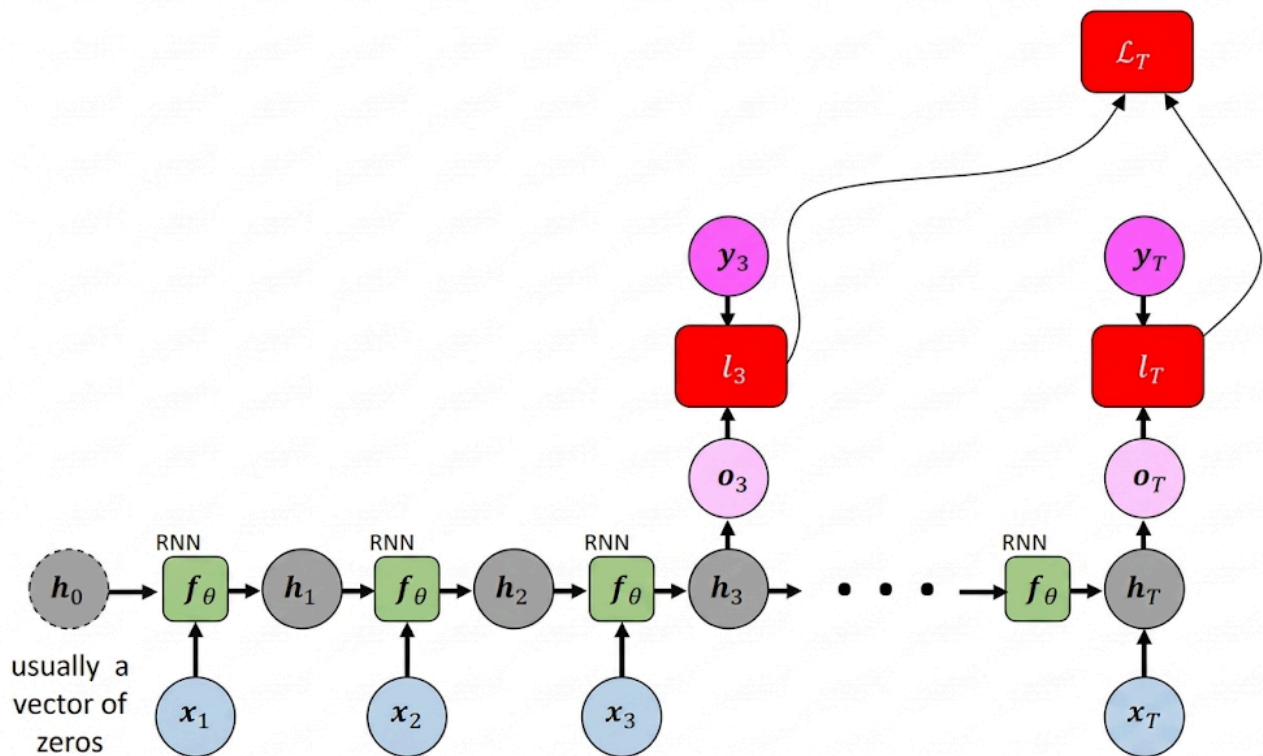


Figura 44: Exemplo de mapeamento many-to-many

- **One-to-many:** Quando o modelo recebe uma única entrada e gera uma sequência de saídas. Por exemplo, uma imagem e a legenda que descreve a imagem.

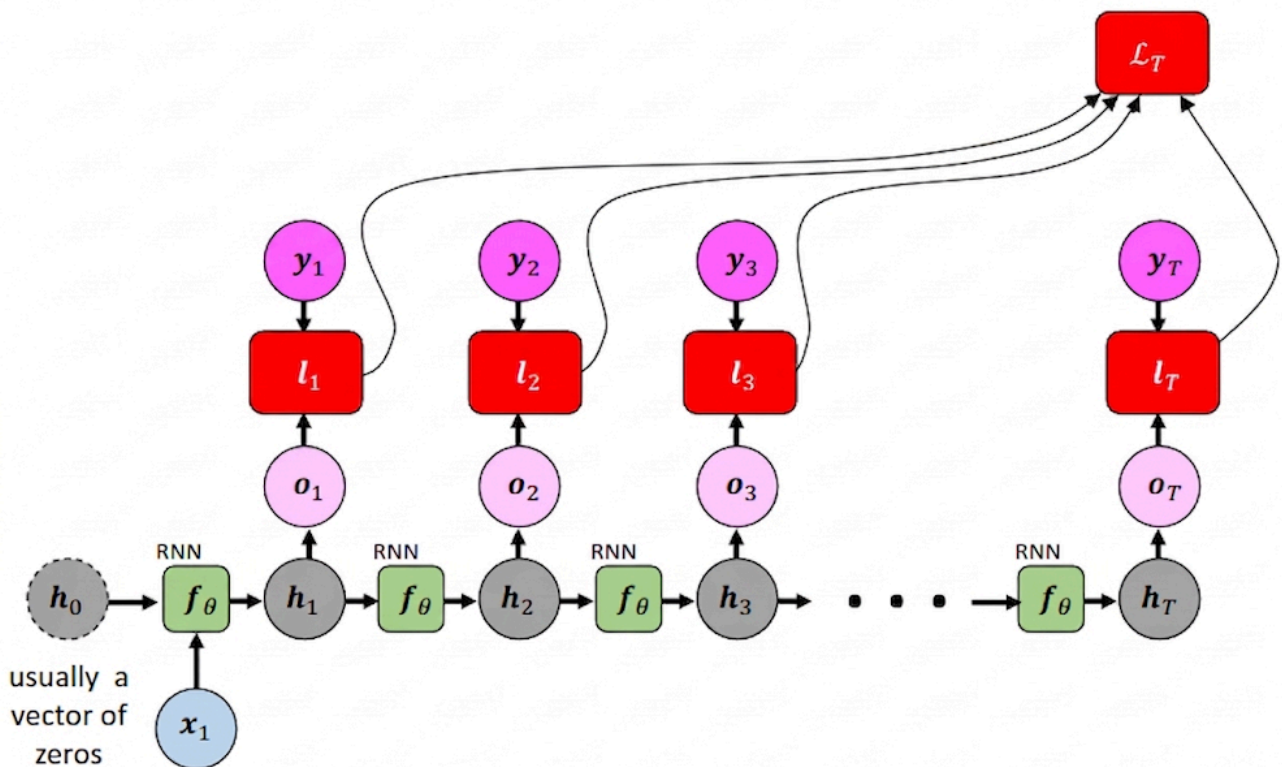


Figura 45: Exemplo de mapeamento one-to-many

- **Many-to-one:** Quando o modelo recebe uma sequência de entradas e gera uma única saída. Por exemplo, uma sequência de palavras e a classificação da sentença.

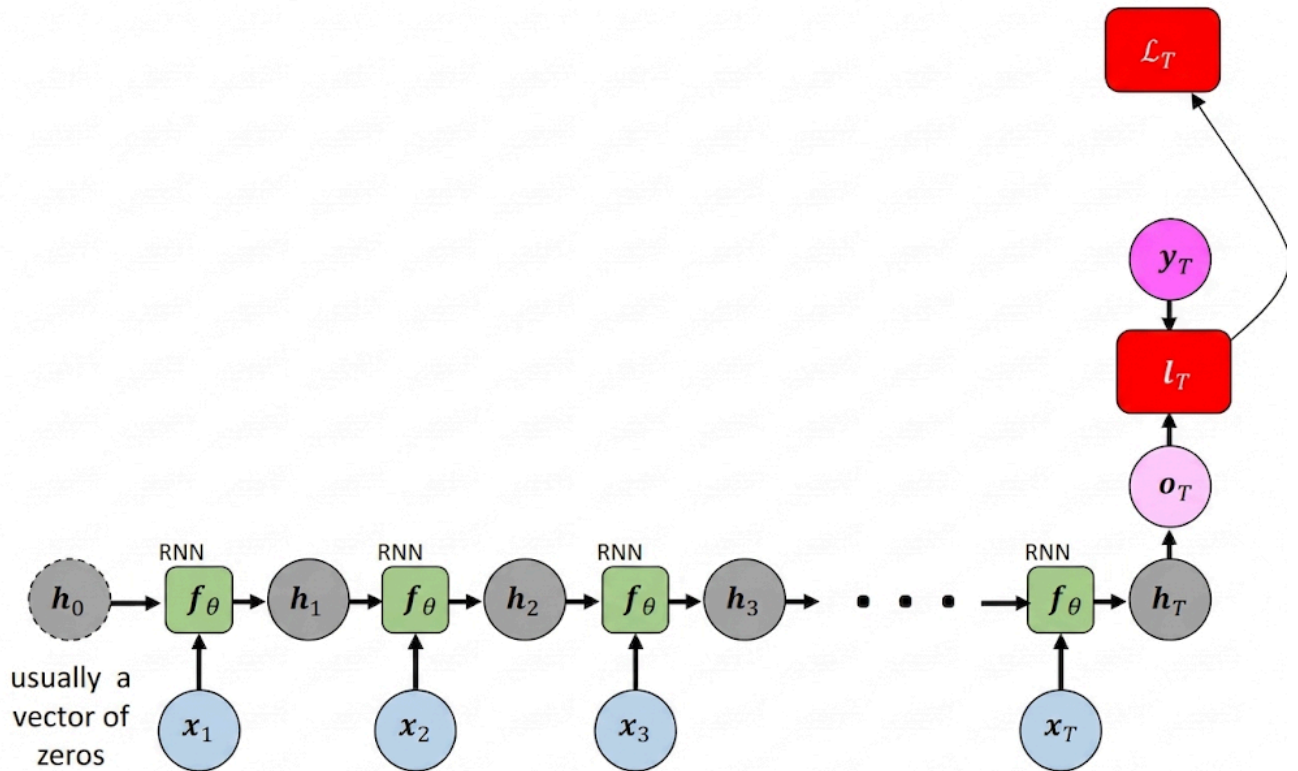


Figura 46: Exemplo de mapeamento many-to-one

3.3 Treinamento e Problemas de Gradiente na RNN

Como comentamos, os mesmos parâmetros θ são utilizados em **todas as camadas** da rede neural, por conta disso, não podemos usar o **backpropagation** tradicional, e sim o **backpropagation through time (BPTT)**, que é uma extensão do algoritmo de retropropagação para redes recorrentes.

3.3.1 Backpropagation Through Time (BPTT)

Como cada célula da RNN é uma camada da rede neural, o processo de unrolling da RNN ao longo do tempo cria uma rede profunda, onde cada passo temporal é tratado como uma camada separada com o diferencial que os mesmos parâmetros são usados em todos os passos.

Em uma tarefa com saída de múltiplos passos temporais (many-to-many), o erro global E é acumulado e calculado a cada instante temporal t

$$E = \sum_{t=1}^T E_t \quad (32)$$

Para atualizar a matriz de pesos compartilhada W_{hh} precisamos calcular a derivada parcial de E em relação a W_{hh} . Pela regra da cadeia, o erro E_t em um determinado instante t depende não apenas da célula no instante t , mas de toda a história de estados ocultos anteriores h_k ($k \leq t$)

$$\frac{\partial E}{\partial W_{hh}} = \sum_{k=1}^T \frac{\partial E_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W_{hh}} \quad (33)$$

As derivadas da esquerda ($\frac{\partial E_t}{\partial h_t}$) e direita ($\frac{\partial h_k}{\partial W_{hh}}$) são fáceis de visualizar pela relação direta que a função derivada tem com o termo da derivação, no entanto, o termo do meio é um pouco mais complexo, mas ele representa **o fluxo do gradiente retropropagado do passo t até o passo k**

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}} \quad (34)$$

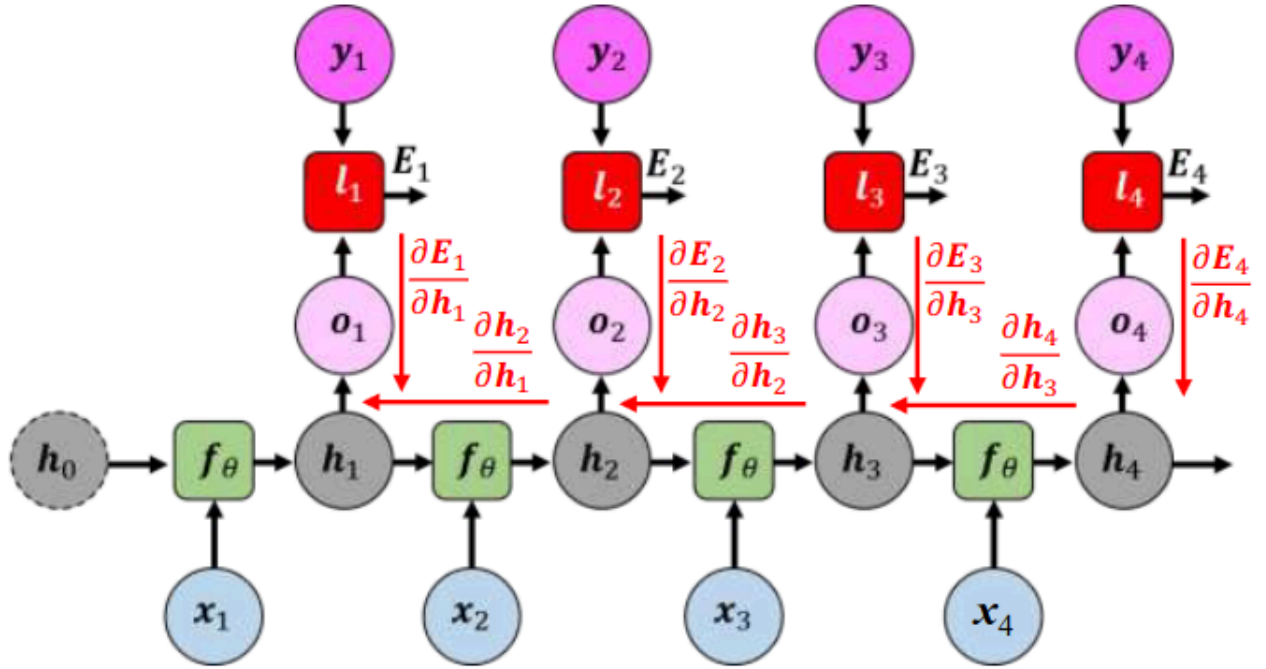


Figura 47: Fluxo do gradiente retropropagado do passo t até o passo k

3.3.2 A matemática dos gradientes explosivos ou desvanecentes

No entanto, essa estrutura pode gerar um grande problema quando a diferença $t - k$ é muito grande. Tomando como base a equação base da RNN $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$, podemos calcular a derivada do estado oculto h_j em relação ao estado oculto anterior h_{j-1}

$$\frac{\partial h_j}{\partial h_{j-1}} = \text{diag}(1 - \tanh^2(\cdot))W_{hh}^T \quad (35)$$

então a propagação do gradiente de h_0 até h_t envolve a multiplicação repetida de $t - k$ termos W_{hh}^T e da derivada de $\tanh$

Podemos chegar nesse mesmo resultado de uma forma mais matemática fazendo uma análise por SVD. Considere a decomposição SVD da matriz de transição $W_{hh} = U\Sigma V^T$ e seja a parcial derivada do estado oculto h_j em relação ao estado oculto anterior h_{j-1} escrita como

$$\frac{\partial h_j}{\partial h_{j-1}} = D_j W_{hh}^T \quad (36)$$

então sabemos que a derivada de h_t com relação a um estado oculto anterior h_k é dada por

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t D_j W_{hh}^T \quad (37)$$

vamos então medir a norma L_2 desse gradiente para entender como ele se comporta

$$\left\| \frac{\partial h_t}{\partial h_k} \right\|_2 = \left\| \prod_{j=k+1}^t D_j W_{hh}^T \right\|_2 \quad (38)$$

Pela propriedade submultiplicativa das normas matriciais $\|AB\|_2 \leq \|A\|_2 \|B\|_2$, temos que:

$$\left\| \frac{\partial h_t}{\partial h_k} \right\|_2 \leq \prod_{j=k+1}^t \|D_j\|_2 \|W_{hh}^T\|_2 \quad (39)$$

No entanto, temos que $\|D_j\|_2 = \sigma_{\max}(D_j) = \max_i |1 - \tanh^2(z_{ji})| \leq 1$ e $\|W_{hh}^T\|_2 = \sigma_{\max}(W_{hh}^T)$, então

$$\begin{aligned} \left\| \frac{\partial h_t}{\partial h_k} \right\|_2 &\leq \prod_{j=k+1}^t 1 \cdot \sigma_{\max}(W_{hh}^T) \\ \Rightarrow \left\| \frac{\partial h_t}{\partial h_k} \right\|_2 &\leq (\sigma_{\max}(W_{hh}^T))^{t-k} \end{aligned} \quad (40)$$

A conclusão que temos dessa análise é que, se o maior valor singular é menor que 1, então o gradiente vai decair exponencialmente com o aumento da diferença $t - k$, levando ao problema de **vanishing gradient**. Por outro lado, se o maior valor singular é maior que 1, então o gradiente vai crescer exponencialmente com o aumento da diferença $t - k$, levando ao problema de **exploding gradient**.

3.3.3 Métodos de mitigação

Para mitigar o problema de **exploding gradients** e **vanishing gradients**. A principal técnica utilizada é uma variação do BPTT.

Truncated BPTT: No BPTT original, para atualizar os pesos, eu faço o forward pass por TODOS os T passos temporais e retropropago por eles novamente. Nessa versão simplificada, existem dois hiperparâmetros k_1 e k_2 . Na parte do forward, a rede propaga por apenas k_1 passos temporais, e na parte do backward, a rede retropropaga por apenas k_2 passos temporais (obrigatoriamente $k_2 < k_1$). Isso reduz a profundidade da rede e ajuda a evitar o problema de gradientes explosivos ou desvanecentes.

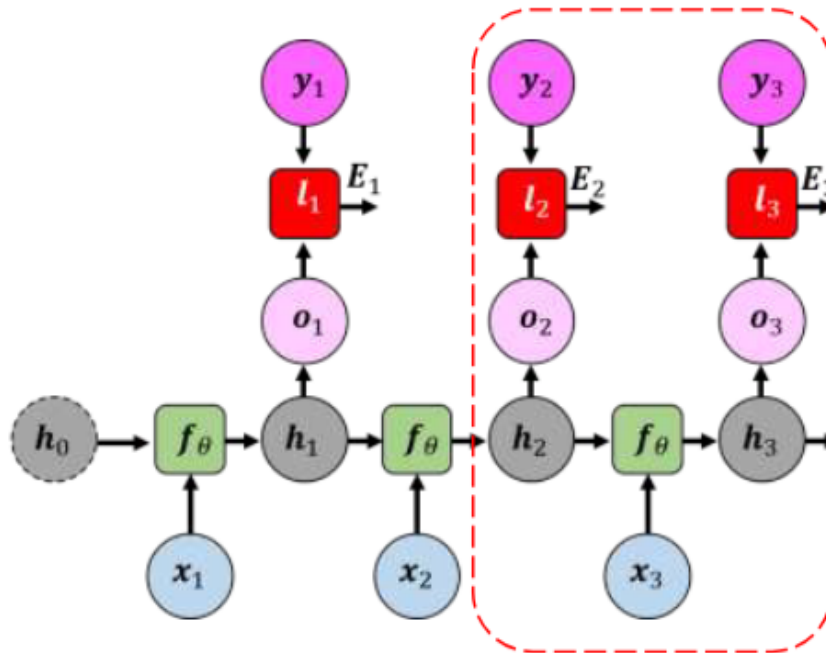


Figura 48: Exemplo de Truncated BPTT com $k_1=3$ e $k_2=2$

3.4 Limitações da RNN

Em RNN simples, ela consegue capturar contexto relevante de curto prazo, mas tem dificuldade em capturar dependências de longo prazo. Isso ocorre porque, à medida que a sequência se torna mais longa, o gradiente pode se tornar muito pequeno (vanishing gradient) ou muito grande (exploding gradient), dificultando o aprendizado de padrões de longo prazo. Por exemplo, na frase “*I grew up in france, [...] I speak fluent french*” as palavras *france* e *french* estão separadas por várias palavras, e a RNN simples pode ter dificuldade em capturar essa relação de longo prazo.

Não só isso, como a RNN também pode ter dificuldade, mesmo em dependências de curto prazo, de selecionar a informação que é de fato relevante para aquele contexto

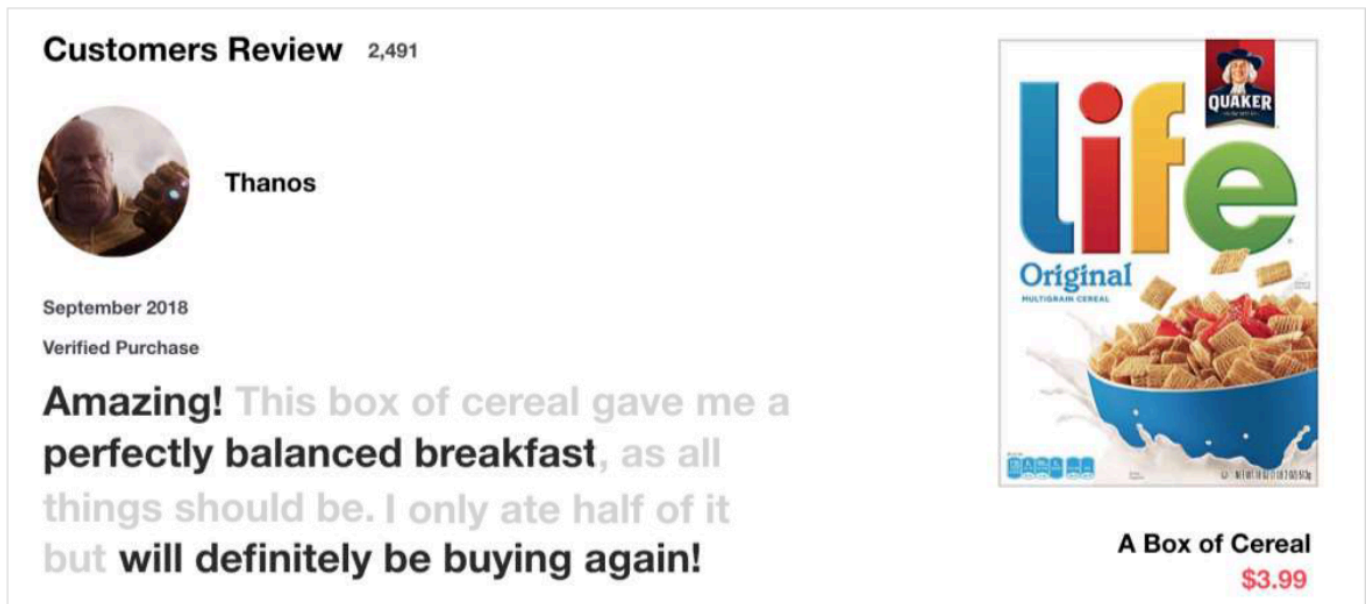


Figura 49: Exemplo de limitação da RNN simples

3.5 Long-short Term Memory (LSTM)

Veio para corrigir as limitações presentes na RNN simples, introduzindo uma arquitetura de célula mais complexa que permite que a rede aprenda a manter ou esquecer informações ao longo do tempo.

Enquanto a RNN simples possui apenas um estado oculto h_t , a LSTM possui dois estados, um estado oculto h_t e um estado de célula c_t que atua como uma “esteira rolante” (**conveyor belt**) que carrega a informação relevante ao longo da sequência temporal com alterações mínimas.

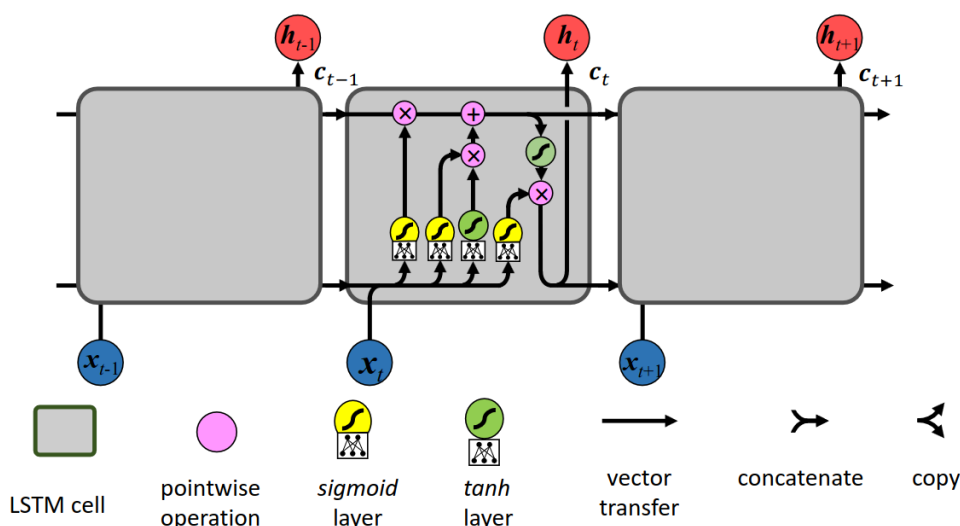


Figura 50: Arquitetura de uma célula LSTM

Podemos estruturar a seguinte comparação: Na RNN Simples, tínhamos que h_t era dado por

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \quad (41)$$

enquanto na LSTM, temos que:

$$\begin{pmatrix} f \\ i \\ s \\ \tilde{c}_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W^T \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \quad (42)$$

$$W = \begin{pmatrix} W_f \\ W_i \\ W_s \\ W_c \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$$

$$h_t = s \odot \tanh(c_t)$$

Mas o que são esse bando de informação extra que a gente adicionou? Para controlar aquilo que entra, permanece e sai do estado celular c_t , a LSTM utiliza de três portas (**gates**), onde cada porta consiste em uma camada de **rede neural** com **ativação sigmoide** combinada com uma operação de multiplicação ponto a ponto (**pointwise**). As portas e componentes são divididos em

- **Porta de esquecimento (f_t):** Quantidade de informação a apagar do estado celular passado
- **Porta do input (i_t):** Quantidade de nova informação a adicionar ao estado celular
- **Valores candidatos ($\tilde{c}_t$):** Valores propostos para serem adicionados ao estado celular
- **Porta de saída/seleção (s_t):** Quantidade do estado celular a revelar como saída no estado oculto

Vamos passar com um pouco mais de calma em cada passo dessa nova arquitetura. Para cada passo temporal t a célula recebe a entrada atual x_t , o estado oculto anterior h_{t-1} e o estado celular anterior c_{t-1} .

Primeiro decidimos quais informações do estado celular anterior c_{t-1} devem ser esquecidas. Isso é feito através da porta de esquecimento f_t , que utiliza uma função sigmoide para gerar valores entre 0 e 1, onde 0 significa “esquecer completamente” e 1 significa “manter completamente”. A equação para a porta de esquecimento é:

$$f_t = \sigma\left(W_f \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_f\right) \quad (43)$$

Então a célula calcula um novo vetor de valores candidatos $\tilde{c}_t$, que contém informações que poderiam ser adicionadas ao estado celular. Isso é feito através de uma função tangente hiperbólica, que gera valores entre -1 e 1. A equação para os valores candidatos é:

$$\tilde{c}_t = \tanh\left(W_c \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_c\right) \quad (44)$$

Então calculamos a porta de input i_t , que decide quais valores candidatos devem ser adicionados ao estado celular. Isso é feito através de uma função sigmoide, que gera valores entre 0 e 1. A equação para a porta de input é:

$$i_t = \sigma \left(W_i \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_i \right) \quad (45)$$

O novo estado celular c_t é calculado combinando a memória antiga filtrada com os novos candidatos filtrados ($\odot$ representa a multiplicação elemento a elemento):

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (46)$$

Depois o portão de seleção vai decidir quais partes do estado celular c_t devem ser reveladas como saída. A porta de seleção s_t é calculada usando uma função sigmoide:

$$s_t = \sigma \left(W_s \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_s \right) \quad (47)$$

então aplicamos s_t ao estado celular c_t passando por uma função tangente hiperbólica para gerar o novo estado oculto h_t :

$$h_t = s_t \odot \tanh(c_t) \quad (48)$$

3.5.1 A inovação no Fluxo do Gradiente

A grande inovação matemática da LSTM que resolve o problema do Gradiente Desaparecido (Vanishing Gradient) está na forma como o erro retropropaga pelo Cell State (c_t)

Na Simple RNN, a derivada do estado em relação ao estado anterior exigia a multiplicação matricial pela transposta dos pesos

$$\frac{\partial h_t}{\partial h_{t-1}} \propto W_{hh}^T \quad (49)$$

Ao retropropagar por $T - k$ passos, acumulava-se a multiplicação de matrizes $(W_{hh}^T)^{T-k}$, fazendo o gradiente desaparecer quando o maior valor singular era menor que 1

Na LSTM, a atualização do estado celular é aditiva: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$. Ao calcular a derivada parcial do estado celular atual c_t diretamente em relação ao estado celular anterior c_{t-1}

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t \quad (50)$$

Ao retropropagar o erro ao longo de uma sequência de T até um instante distante k exclusivamente pela linha do Cell State, o gradiente resultante é dado por

$$\frac{\partial c_T}{\partial c_k} = \prod_{j=k+1}^T f_j \quad (51)$$

Sem Multiplicação Matricial Sucessiva: A retropropagação de c_{t-1} para c_t envolve apenas multiplicação elemento a elemento pelo vetor do forget gate f_t , eliminando a multiplicação matricial por W^T . Como cada passo temporal possui um vetor f_j diferente (calculado dinamicamente com base em h_{j-1} e x_j), a rede pode aprender a definir $f_j \approx 1.0$ para manter memórias importantes intactas. Desta forma, a informação e o gradiente conseguem fluir sem desaparecer ao longo da “esteira” do Cell State por centenas de passos temporais.

3.6 Gated Recurrent Unit (GRU)

A **Gated Recurrent Unit (GRU)** foi introduzida em 2014 por Cho et al. como uma variação simplificada da LSTM, também projetada para evitar o problema das dependências de longo prazo (**long-term dependency problem**).

Por ter uma estrutura mais enxuta, a GRU possui menos parâmetros e é ligeiramente mais rápida de treinar do que a LSTM

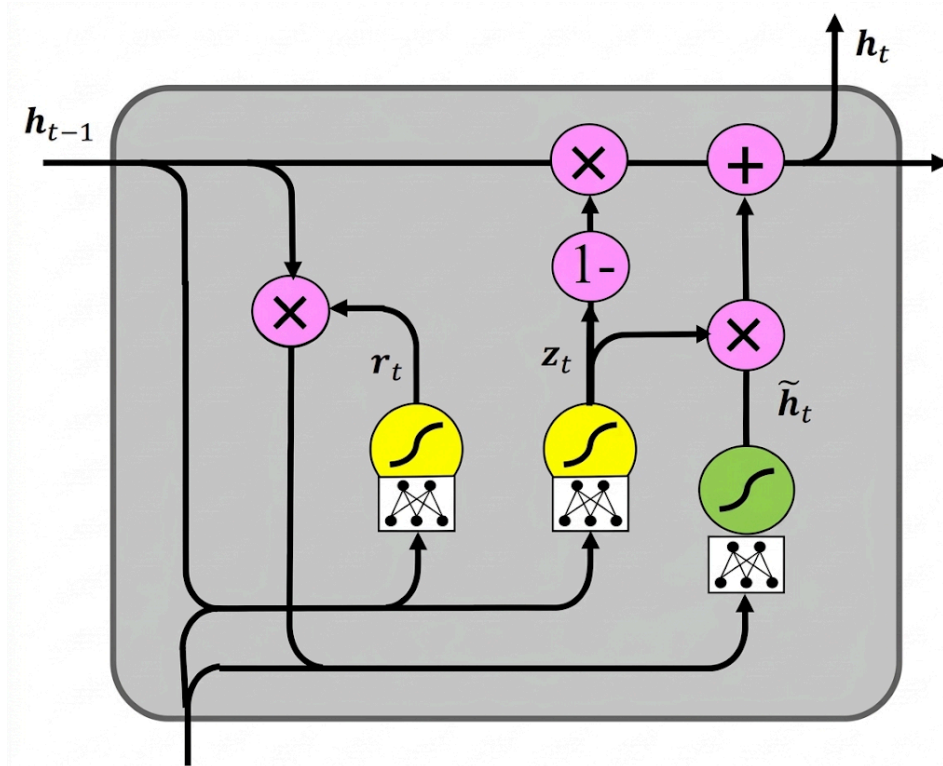


Figura 51: Arquitetura de uma célula GRU

As principais diferenças que ocorrem são a **eliminação do cell state** c_t e utilizamos apenas o estado oculto h_t e agora são 2 portas em vez de 3

- **Porta de atualização (z_t):** Determina quanto do novo estado oculto deve ser atualizado com base no novo input
- **Porta de redefinição (r_t):** Controla quanto do estado oculto anterior deve ser usado para calcular o novo estado oculto

Para cada passo temporal t , dada a entrada x_t e o estado oculto anterior h_{t-1} , os gates são calculados como:

$$\begin{aligned} z_t &= \sigma \left(W_z \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_z \right) \\ r_t &= \sigma \left(W_r \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} + b_r \right) \end{aligned} \quad (52)$$

Depois o estado oculto candidato é calculado utilizando a porta de redefinição r_t para controlar a influência do estado oculto anterior:

$$\tilde{h}_t = \tanh \left(W_h \begin{pmatrix} r_t \odot h_{t-1} \\ x_t \end{pmatrix} + b_h \right) \quad (53)$$

E finalmente atualizamos e geramos o novo estado oculto h_t utilizando a porta de atualização z_t :

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \quad (54)$$

3.7 Aplicando CNN

Podemos querer utilizar as RNNs para processar sequências de imagens, como em vídeos, onde cada frame é uma imagem. Nesse caso, podemos combinar Convolutional Neural Networks (CNNs)

com RNNs para extrair características espaciais das imagens e capturar dependências temporais entre os frames.

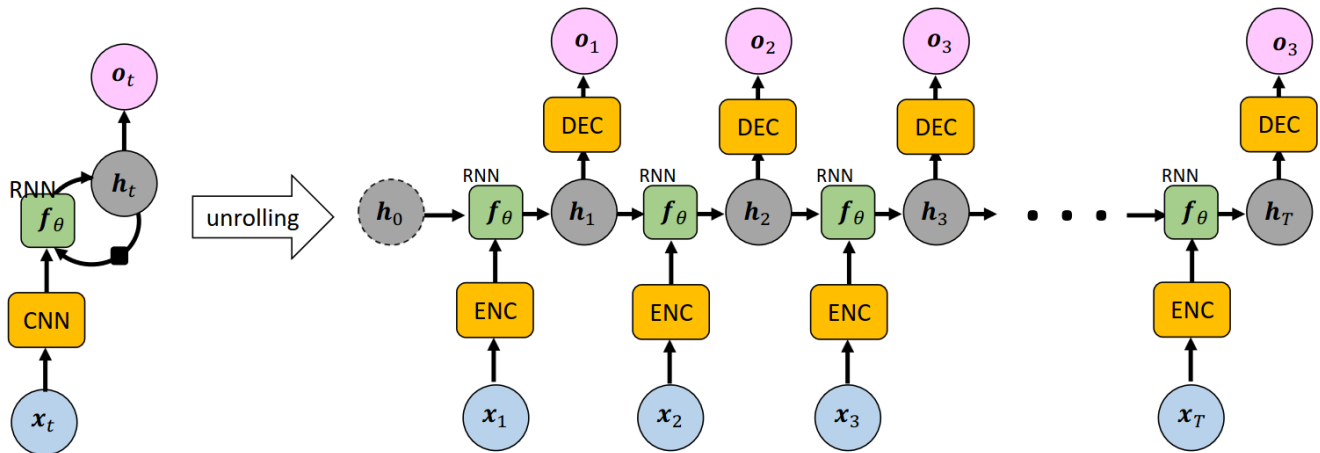


Figura 52: Arquitetura de uma CNN seguida por uma RNN

O encoder fica responsável por extrair características espaciais de cada frame do vídeo, enquanto a RNN processa essas características ao longo do tempo para capturar a dinâmica temporal do vídeo, já o decoder fica responsável por gerar a saída final. Essa abordagem é útil em tarefas como reconhecimento de ações em vídeos, onde é importante entender tanto o conteúdo visual de cada frame quanto a sequência de eventos ao longo do tempo.

3.8 RNN Bidirecionais

As RNN padrão utilizam apenas informação do **passado** para prever a saída atual, mas em muitas tarefas, a informação do **futuro** também pode ser útil. As RNN bidirecionais (BRNNs) processam a sequência de dados em duas direções: uma RNN lê a sequência do início ao fim (forward), enquanto outra RNN lê a sequência do fim ao início (backward). As saídas das duas RNNs são então combinadas para formar a saída final.

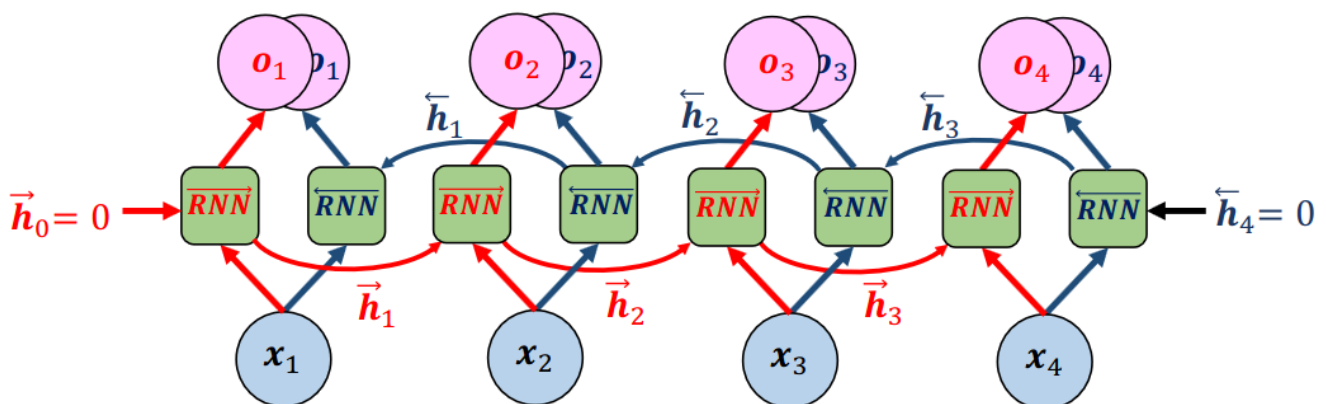


Figura 53: Arquitetura de uma RNN Bidirecional

Generative Adversarial Networks (GANs)

